

ORIGINAL ARTICLE OPEN ACCESS

Event-Driven Computing Continuum Orchestration Solver (ECCOS): A Discrete-Event Simulation Framework for Optimal Service Placement and ML-Ready Trace Generation

Mireia Jaume  | Isaac Lera  | Carlos Guerrero 

Computer Science Department, University of Balearic Islands, Palma, Spain |

Correspondence: Carlos Guerrero (carlos.guerrero@uib.es) | Isaac Lera (isaac.lera@uib.es)**Received:** xx January 20xx | **Revised:** xx January 20xx | **Accepted:** xx January 20xx**Academic Editor:** TBD | **Guest Editor:** TBD**Funding:** Grant PID2024-158637OB-I00, funded by MICIU/AEI/10.13039/501100011033 and by “ERDF A way of making Europe”, “ERDF/EU”.**Keywords:** Computing Continuum | Discrete-Event Simulation | Synthetic Machine Learning Ready Datasets | Service Placement Problem | Resource Optimization

ABSTRACT

The Computing Continuum (Cloud–Fog–Edge) paradigm requires real-time, adaptive service orchestration across volatile, highly heterogeneous, and spatially distributed execution environments. However, the design and evaluation of data-driven orchestration algorithms—particularly Reinforcement Learning (RL) and Graph Neural Network (GNN) policies—are severely hindered by the scarcity of large-scale production traces and the limitations of traditional discrete-event simulators, which often suffer from computational packet-level bottlenecks, monolithic coupling, and static scenario modeling. To bridge this gap, this paper introduces ECCOS (Event-Driven Computing Continuum Orchestration Solver), an open-source macroscopic discrete-event simulation framework engineered for optimal service placement and automated Machine Learning (ML)-ready trace generation. ECCOS employs a strictly decoupled architecture comprising context configuration (ECCO-ARCHITECT), discrete-event stochastic scheduling (ECCO-GENERATOR), authoritative state accounting (ECCO-STATE), stateless placement optimization (ECCO-SOLVER), and data integration (ECCO-BRIDGE). The framework supports declarative YAML-based scenario specifications, domain-isolated pseudorandom generator streams for deterministic experimental reproducibility, dynamic microservice graph mutations (SFCs and DAGs), and user-composable macro-events that capture complex physical disruptions. Crucially, ECCOS automatically formats orchestration cycles into non-Euclidean, self-contained JSON trace records mapping onto Markov Decision Process (MDP) transition tuples. Experimental evaluations under severe stress scenarios, including regional electrical storms, localized mass gathering and viral demand surges, demonstrate the capacity of ECCOS to capture complex spatiotemporal dynamics and benchmark multi-objective placement strategies without introducing computational bottlenecks.

1 | Introduction

The Computing Continuum (Cloud-Fog-Edge) paradigm has emerged as the foundational infrastructure to support the next generation of ultra-low latency and bandwidth-intensive applications, such as distributed Artificial Intelligence (AI), 5G/6G networks, and the Internet of Things (IoT) [1]. Unlike traditional centralized data centers, the Continuum is characterized by extreme heterogeneity, resource scarcity at the Far-Edge, and high spatiotemporal volatility in both the underlying infrastructure and user demand [2]. In this highly dynamic environment, traditional heuristic approaches for service placement may be insufficient on their own. Consequently, the scientific community has extended and complemented existing optimization strategies with Machine Learning techniques, particularly Reinforcement Learning (RL), to enable more adaptive, real-time orchestration [3].

However, the advancement of AI applied to Continuum orchestration is hindered by a critical gap in the State of the Art, the scarcity of dynamic evaluation environments and the lack of large-scale training datasets [4]. Real-world production traces remain inaccessible due to corporate privacy policies, and existing simulation tools (such as ns-3, iFogSim, or CloudSim) exhibit fundamental shortcomings [5, 6]. First, traditional network simulators operate at the packet level, rendering them computationally unscalable for generating the months of macroscopic data required to train a Machine Learning model. Second, they lack granular and reproducible configurations, forcing researchers to hardcode scenarios directly into the source code. Third, they present a tight coupling between the simulation engine and the optimization policies, hindering the integration of new algorithms or external agents. Finally, they model system volatility in a binary manner (e.g., assuming either perfect operation or total permanent failure), completely ignoring complex transient phenomena such as partial capacity degradations (brownouts), viral demand spikes (flash crowds), or runtime architectural mutations.

To address these challenges, this paper presents ECCOS (*Event-Driven Computing Continuum Orchestration Solver*), a macroscopic simulation environment based on discrete events. ECCOS is explicitly designed from its inception to abstract low-level network complexity and focus on the agile, stochastic, and mathematically grounded generation of stress scenarios. Its primary objective is to provide the scientific community with a standardized tool for evaluating orchestration algorithms and generating massive, labeled synthetic datasets.

To tackle the aforementioned limitations, the main contributions of this work are summarized as follows:

- **Scalable Dataset Generation for Machine Learning:** ECCOS natively produces output traces rich in state variables and metrics, explicitly oriented toward the creation of massive datasets for the continuous training of Machine Learning algorithms (both supervised and RL).
- **Modular Architecture and Decoupled Solvers:** A strictly decoupled design is proposed, separating logical modules (context creation, event engine, state manager, and optimization engine). ECCOS includes several native solvers (Integer Linear Programming, ILP, Greedy heuristics...), but its agnostic interface allows researchers to easily implement and plug in their own custom solvers without altering the framework's core.
- **Fine-grained User Definition via YAML:** A declarative configuration system using YAML files is introduced, enabling users to define hybrid topologies, probabilistic distributions, and system behaviors with extreme granularity, thereby guaranteeing total experimental reproducibility.
- **Rigorous Stochastic Modeling and Event Composition:** The framework features an extensive catalog of configurable events that model real scenario entropy (such as brownouts, temporal user disconnections, and geographical demand shifts). Furthermore, its architecture allows for the seamless composition of new, complex events from the natively implemented base events.
- **Dynamism in the Application Lifecycle:** ECCOS allows the simulation of “Day 2” operations, enabling runtime architectural mutations that dynamically alter both the topology and the resource footprint of instantiated microservice chains.

2 | State of the Art

Over the last decade, numerous simulation frameworks have been developed to model the complex interactions within the IoT-Fog-Cloud continuum. While these tools have provided foundational insights into resource management, latency optimization, and service placement, they present significant structural and functional limitations when addressing modern, data-driven, and highly dynamic microservice architectures [7].

The current landscape of fog simulators is dominated by discrete-event simulation models, many of which are extensions of the widely used CloudSim toolkit [8]. Prominent examples include iFogSim [9], EdgeCloudSim [10], and

PureEdgeSim [11]. Other notable independent frameworks include YAFS (Yet Another Fog Simulator) [12], FogNet-Sim++ [13], and FogTorchII [14]. Despite their utility, an in-depth analysis of these state-of-the-art frameworks reveals critical shortcomings across several dimensions, which the proposed ECCOS simulator explicitly addresses.

Scalable Dataset Generation for Machine Learning. Modern resource allocation and orchestration strategies increasingly rely on Machine Learning (ML) and Reinforcement Learning (RL) algorithms. However, traditional simulators are not designed with data generation as a primary objective. Simulators such as EdgeCloudSim and YAFS typically export performance metrics via Comma Separated Values (CSV) files intended for post-simulation statistical analysis [15]. They fail to natively capture the deep, continuous state-action-reward interactions required for RL training. ECCOS overcomes this limitation by natively producing rich output traces loaded with granular state variables and metrics. It is explicitly oriented toward the automated creation of massive, high-dimensional datasets, directly bridging the gap between network simulation and continuous ML algorithm training.

Modular Architecture and Decoupled Solvers. A major drawback of prevalent tools like iFogSim is their monolithic design, which results in a steep learning curve and complicates the implementation of custom algorithms [16]. Modifying resource management policies in these simulators often requires invasive changes to the core framework code. While tools like FogWorkflowSim attempt to simplify task scheduling evaluation [17], they still lack a strictly decoupled architecture. ECCOS introduces a highly modular design that strictly separates logical components (context creation, event engine, state manager, and optimization engine). Unlike existing tools, ECCOS acts through an agnostic interface that natively includes several solvers while allowing researchers to seamlessly plug in custom solvers without altering the framework's core logic.

Fine-grained User Definition and Reproducibility. The configuration of physical topologies, user behaviors, and application definitions in current simulators is often rigid and fragmented. Simulators like DISSECT-CF-Fog and PureEdgeSim rely on verbose XML files while YAFS and iFogSim utilize JSON or require hardcoded Java/Python programmatic setup. These approaches lack the declarative simplicity and precision needed for complex scenario modeling. ECCOS pioneers a declarative configuration system using YAML files, enabling researchers to define hybrid topologies, probabilistic distributions, and fine-grained system behaviors. This level of extreme granularity not only simplifies the setup of complex scenarios but also guarantees total experimental reproducibility.

Rigorous Stochastic Modeling and Event Composition. Real-world fog environments are characterized by high entropy and volatility. Existing simulators offer limited stochastic modeling; for instance, YAFS supports network failures and node disconnections and FogTorchII uses Monte Carlo methods to simulate deviations in Quality of Service (QoS) profiles [18]. However, these frameworks do not allow for the modeling of compounding, complex network phenomena. ECCOS addresses this gap by featuring an extensive catalog of configurable base events that accurately model real network entropy, including temporal user disconnections, localized brownouts, and geographical demand shifts. Furthermore, its architecture supports the seamless composition of new, complex events from these native bases, providing a much more rigorous and realistic stochastic modeling environment than currently available.

Dynamism in the Application Lifecycle ("Day 2" Operations). A critical shortcoming in the current fog simulation literature is the static treatment of application lifecycles. Most simulators, including iFogSim and FogTorch, treat application deployment as a static placement problem. Advanced tools like MobFogSim [19] introduce mobility and basic Virtual Machine (VM)/container migration, but they fundamentally maintain the original architecture of the deployed service [20]. They lack the capability to model runtime elasticity and architectural evolution. ECCOS distinguishes itself by simulating true "Day 2" operations. It enables runtime architectural mutations, allowing the simulator to dynamically alter both the network topology and the resource footprint of instantiated microservice chains, accurately reflecting the lifecycle dynamism of modern cloud-native applications operating at the edge.

3 | ECCOS Simulator Architecture and Modular Design

We present ECCOS (*Event-Driven Computing Continuum Orchestration Solver*) to overcome the architectural rigidity of traditional edge simulation tools, which frequently interleave stochastic event generation, state accounting, and placement optimization into monolithic execution loops. As illustrated in Figure 1, ECCOS employs a modular, highly decoupled design pattern structured around four foundational components: ECCO-ARCHITECT, ECCO-GENERATOR, ECCO-STATE, ECCO-SOLVER, and ECCO-BRIDGE. This separation of concerns establishes strict data interfaces and transactional protocols between functional phases, ensuring reproducibility, extensible resource accounting, and pluggable optimization.

The lifecycle of an ECCOS simulation is governed by structured data exchanges and distinct operational responsibilities across its core conceptual modules:

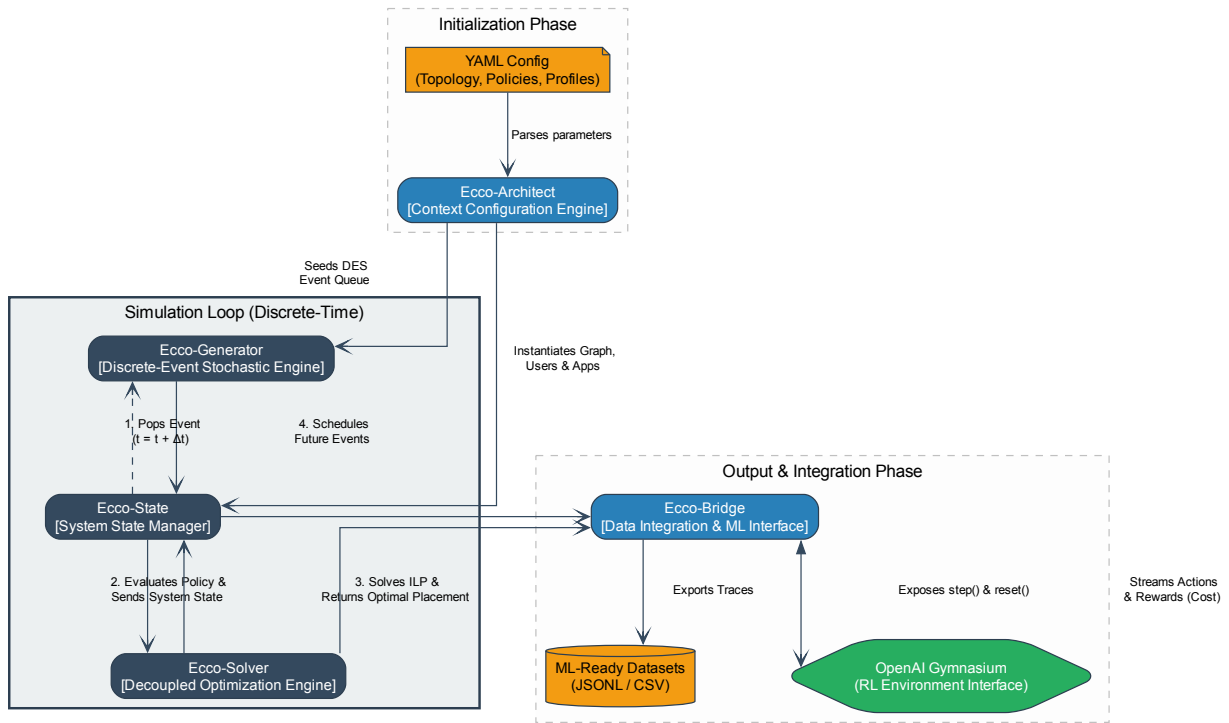


FIGURE 1 | Modular architecture and transactional data flows of the ECCOS simulation framework, highlighting the decoupled interaction between declarative initialization, discrete-event scheduling, state management, and placement optimization.

- **ECCO-ARCHITECT (Context Configuration Engine):** Operating during the *Initialization Phase*, ECCO-ARCHITECT ingests declarative user manifests written in YAML (Config). It parses topological definitions, user mobility/disconnection models, and application topologies and profiles to instantiate the foundational data structures: the multi-attribute infrastructure graph (InfrastructureSet), the user population (UserSet), and the microservice DAGs (AppSet). Once validated, ECCO-ARCHITECT seeds the initial event chronology into ECCO-GENERATOR and transfers the instantiated structural references to ECCO-STATE.
- **ECCO-GENERATOR (Discrete-Event Stochastic Engine):** Acting as the chronological scheduler of the *Discrete-Time Simulation Loop*, ECCO-GENERATOR maintains the priority queue of stochastic simulation events (e.g., user spatial mobility, transient disconnections, popularity surges, and infrastructure node/edge failures). At each discrete temporal advancement $t = t + \Delta t$, it pops the earliest scheduled event payload and delivers it directly to ECCO-STATE for structural execution.
- **ECCO-STATE (System State Manager):** ECCO-STATE serves as the authoritative *Single Source of Truth* and structural execution handler. Upon receiving an event from ECCO-GENERATOR, it mutates the underlying data structures accordingly, updating user coordinates, toggling edge/node operational flags, and adjusting microservice demand profiles. It continuously enforces schema-agnostic resource accounting across static residency attributes and dynamic load attributes. Guided by user-defined trigger policies (trigger_policy), ECCO-STATE decides whether to absorb event mutations internally or to fire an optimization request. When triggered, it delivers an immutable, consolidated system state snapshot $\mathcal{S}^{(t)} = (\mathcal{G}^{(t)}, \mathcal{A}^{(t)}, \mathcal{U}^{(t)})$ to ECCO-SOLVER.
- **ECCO-SOLVER (Decoupled Optimization Engine):** Operating as a stateless decision engine, ECCO-SOLVER receives the canonical snapshot $\mathcal{S}^{(t)}$ and formulates the optimal placement problem. Regardless of the underlying algorithmic strategy, the solver computes an optimal mapping of microservices onto physical nodes. It returns a structured placement dictionary back to ECCO-STATE, which transactionally applies the decision to the persistent infrastructure graph.
- **ECCO-BRIDGE (Data Integration & ML Interface):** Operating across the *Output and Integration Phase*, ECCO-BRIDGE acts as the unified telemetry exporter and artificial intelligence interoperability layer. Throughout the

simulation loop, it asynchronously consumes two complementary data streams: raw infrastructure telemetry and state observations streamed from ECCO-STATE, alongside placement decisions, execution latencies, and reward/cost metrics emitted by ECCO-SOLVER. ECCO-BRIDGE decouples internal simulation structures from external analytics by supporting both offline dataset serialization and online agent interoperability. In the offline mode, it compiles structured, time-stamped execution traces into standardized, ML-ready formats, facilitating benchmarking, offline regression, and offline Reinforcement Learning training.

The simulation framework was fully implemented in Python (version 3.13.2), leveraging its robust ecosystem for scientific computing and system modeling. The core logic and internal data structures were developed using standard built-in libraries, whereas intensive data manipulation and numerical computations were optimized through the integration of NumPy (v2.4.4) and Pandas (v3.0.3). To accurately model and analyze the underlying network topologies and complex entity interactions within the simulated infrastructure, the NetworkX library (v3.6.1) was extensively employed. Furthermore, algorithmic decision-making processes, such as resource allocation and service placement tasks, were mathematically formulated and solved utilizing the PuLP optimization framework (v3.3.0). The source code is openly available in a GitHub repository¹, which also hosts the result traces generated during the experiments reported in this article (Section 4).

The following subsections describe each of these modules in detail, outlining their internal responsibilities, configuration mechanisms, and role within the overall ECCOS simulation workflow.

3.1 | Ecco-Architect: Context Configuration Engine

The ECCO-ARCHITECT module defines the static context from which each ECCOS simulation scenario is instantiated. Through a declarative YAML configuration, users specify the structural and stochastic models that describe the Computing Continuum infrastructure, the spatial distribution and mobility behavior of users, and the composition, resource demands, and popularity of applications [21]. This module therefore acts as the scenario-construction layer of the simulator: it generates the initial infrastructure graph, assigns physical and logical attributes to nodes and links, creates the initial user population and mobility profiles, and defines the application workload model used later by the event generator and placement solver.

Many configuration aspects require users to specify statistical distributions together with their corresponding parameters, including resource capacities, mobility properties, workload demands, popularity values, and event inter-arrival times [22, 23, 24]. The complete set of distributions currently supported by the simulator, together with their expected YAML parametrization, is provided in the corresponding appendix (Section A).

To ensure strict experimental repeatability and prevent stochastic stream interference across architectural layers, the ECCOS framework isolates pseudorandom number generation (PRNG) into dedicated, independently seeded streams for each structural domain: the physical network topology, the application ecosystem, and the user population. Relying on a shared global random state, a procedural adjustments in one component shift the pseudorandom sequence of downstream processes. Thus, each entity set is governed by an autonomous NumPy Generator instance in ECCOS implementation. This domain decoupling supports deterministic repeatability under fixed seeds and execution order across simulation executions and enables combinatorial benchmarking [25, 26]. For instance, researchers can generate diverse user mobility profiles or application demand landscapes over an identical underlying physical network topology without cross-contamination, ensuring that comparative structural evaluations are strictly controlled and reproducible.

3.1.1 | Infrastructure Definition

Figure 2 summarizes the UML model that supports the infrastructure configuration described in this subsection. The discussion below focuses on the main modeling decisions and their role within ECCOS, while the corresponding appendix provides a detailed description of the YAML parameters used to instantiate these infrastructure models (Section B.1). The infrastructure definition in ECCO-ARCHITECT is driven by the statistical and random models used to construct the graph that represents the Computing Continuum. In this graph, devices are modeled as nodes, while network links among them are represented as edges [27]. Both types of elements are subsequently annotated with configurable dynamic attributes that capture the relevant physical and operational characteristics of real infrastructures, such as computational capacity, memory, latency, bandwidth, or any other resource-related descriptor required by the placement and orchestration logic [28].

¹ <https://github.com/XXXXXX>

The first step in this process is to specify the number of nodes to be generated and the random or statistical model that determines the structural shape of the graph [29]. This choice defines how nodes are interconnected, the resulting topological properties of the simulated infrastructure, and, in combination with the layer-assignment strategy, how each node is mapped onto the cloud, fog, or edge segments of the Computing Continuum. In some cases, this logical hierarchy is produced implicitly during graph construction, whereas in others it is derived afterwards using a model-specific mapping strategy. Consequently, topology generation and layer assignment are complementary mechanisms that jointly determine both the physical connectivity structure and the logical continuum hierarchy [30]. ECCOS provides five graph-generation models to represent the heterogeneity and geographical distribution of Computing Continuum infrastructures:

1. **Scale-Free Topology.** [31] This model is designed to emulate the backbone of the Internet and large-scale telecommunication networks, where a few central nodes act as massive communication hubs. The topology is generated using the Barabási-Albert (BA) model, which relies on a preferential attachment mechanism governed by the number of edges a new node establishes with existing nodes. A centrality-based classification, within user-defined thresholds, are employed to assign nodes to the three layers of the continuum. Nodes exhibiting the highest centrality scores are designated as the Cloud tier, intermediate ones form the Fog, and the vast majority of peripheral nodes with low centrality compose the Edge tier.
2. **Random Graph (Erdős-Rényi) Topology.** [29] The simulator implements the Erdős-Rényi (ER) probabilistic model to baseline network connectivity and simulate unstructured, highly random ad-hoc meshes infrastructures. In this approach, every potential edge between any two nodes is created independently with a uniform probability. Since this model lacks an inherent hierarchical structure or distinct hubs, the logical mapping to the Computing Continuum also relies on Betweenness Centrality thresholds.
3. **Spatial (Geometric) Topology.** [32] Aimed at simulating wireless sensor networks, vehicular ad-hoc networks (VANETs), and Far-Edge environments, this model utilizes Random Geometric Graphs. Nodes are uniformly distributed across a two-dimensional Euclidean plane and are interconnected if the distance between them falls below a specified communication radius. The continuum layer assignment also relies on the Betweenness Centrality of the resulting physical graph. Nodes that serendipitously bridge multiple geographical clusters acquire high centrality and are classified as Cloud or Fog gateways, whereas the scattered exterior nodes remain as Edge devices.
4. **Hierarchical Fat-Tree Topology.** [33] For scenarios focused on modern intra-datacenter networks or highly structured corporate infrastructures, the simulator implements perfectly balanced Tree and highly-redundant Fat-Tree graphs. These topologies are strictly defined by their branching factor and total height. The mapping to the logical continuum is purely deterministic and depth-based (number of hops from the root). The root nodes inherently represents the Cloud datacenter, the intermediate aggregation layers act as Fog switches, and the bottom-tier leaf nodes are strictly designated as the Edge layer.
5. **Multi-Tier Hybrid Topology.** [30, 34] To represent the most comprehensive and realistic end-to-end Computing Continuum ecosystem, the simulator introduces a novel composite topology. This model intrinsically builds and stitches three independent subgraphs: (a) Complete Graph to represent the highly interconnected Cloud core; (b) A Barabási-Albert Scale-Free Graph to model the regional Fog gateways; (c) A Random Geometric Graph to simulate the densely populated Edge devices. Once generated, the tiers are geographically synthesized and cross-linked using Euclidean distance minimization, stitching Edge nodes to their closest Fog gateway, and Fog nodes to their nearest Cloud counterpart. In this paradigm, the logical mapping is intrinsic to the generation process, ensuring a structurally robust Cloud-Fog-Edge hierarchy by design.

To realistically characterize the generated nodes and edges, ECCOS relies on configurable attributes that describe the resources and operational properties associated with each infrastructure element. These attributes may be defined for nodes, or edges, and the user can specify as many attributes as required by the target scenario, such as CPU capacity, memory, storage, latency, bandwidth, energy availability, or reliability indicators [35]. For each attribute, the configuration must define how its initial values are assigned across the generated topology.

Two basic and complementary initialization strategies are supported for those attributes. In the first one, attribute values are sampled independently for each element, regardless of its position in the graph. This mode is appropriate when resources are assumed to be randomly distributed across the infrastructure [36]. In the second strategy, the assignment is topology-aware and depends on the centrality of each element, so that the structural role of a node or edge influences the resources it receives [37]. In this case, the centrality value can be used in a direct proportional manner, multiplying the sampled resource values by the centrality score and therefore assigning larger capacities to elements closer to the cloud or to highly central positions. Conversely, an inverse proportional mode multiplies the sampled values by the inverse centrality term, $(1 - \text{centrality})$, producing lower resource values for elements close to the cloud and higher values toward less central regions. The base resource values are generated from an explicit statistical distribution, ensuring that both randomness and topology-dependent structure can be combined in a controlled and reproducible way. In addition,

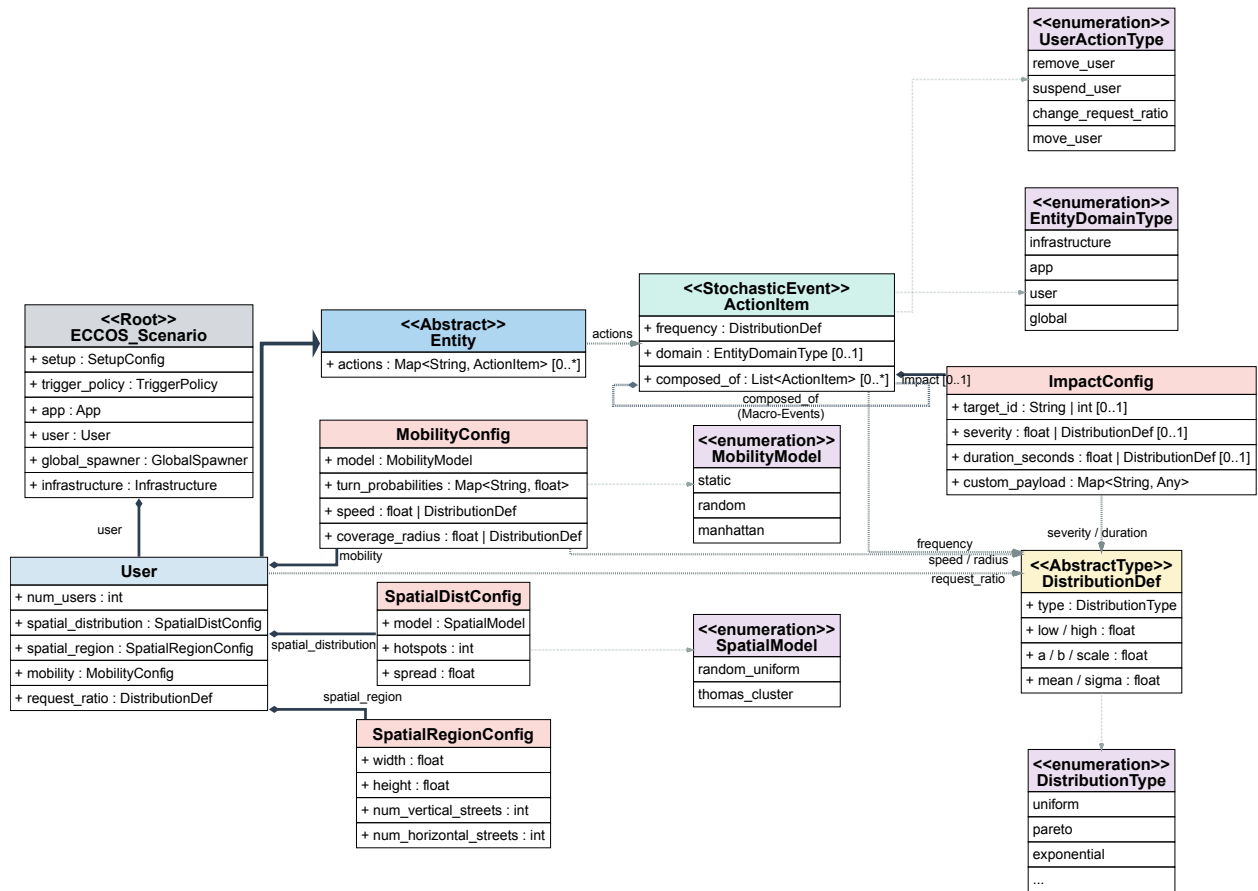


FIGURE 3 | UML model for user definition.

user consumes multiple applications can be emulated straightforwardly by generating a larger population of user entities, each representing one user–application demand relationship.

ECCOS supports user mobility during the simulation. To enable this behavior, the configuration must define the physical space in which users move, their initial positions, and the mobility pattern that determines how they change location over time.

The physical space is defined by a two-dimensional plane, with configurable width and height parameters. In addition, the number of equally spaced streets is also user-defined for the specific case of the Manhattan user mobility model.

For the initial placement of the users in the plane, a first simple spatial distribution model offers a purely stochastic random uniform distribution, which isotropically spawns users across the entire 2D topology space with equal probability [40]. This model serves as a fundamental baseline, emulating widely dispersed and homogeneous user scenarios, such as suburban neighborhoods or rural deployments, where no significant aggregation factors influence network utilization. But in real-world urban scenarios, mobile users do not distribute homogeneously across the topology, they tend to aggregate around specific centers of interest such as transit hubs, shopping malls, or stadiums [41]. The simulator implements a second placement model that considers a stochastic Thomas Cluster Process. This spatial model first generates a predefined number of latent "parent points" (or hotspots) across the simulated 2D plane, with their spatial dispersion sampled from a two-dimensional normal distribution. Once these hotspots are established, individual users are assigned to them and their exact coordinates are instantiated using an isotropic two-dimensional Gaussian distribution centered at their respective hotspot [42]. The degree of spatial aggregation is strictly controlled by a variance parameter, which dictates the dispersion of the user population. A tight spread generates highly dense crowds inherently constrained to the hotspot core, whereas a larger spread models a more relaxed distribution, similar to users roaming the surrounding neighborhoods of a central landmark.

To accurately emulate the inherent heterogeneity of mobile environments, the simulation framework dynamically individualizes the kinetic profile of each user entity during initialization [43]. The traversal speed and the maximum coverage

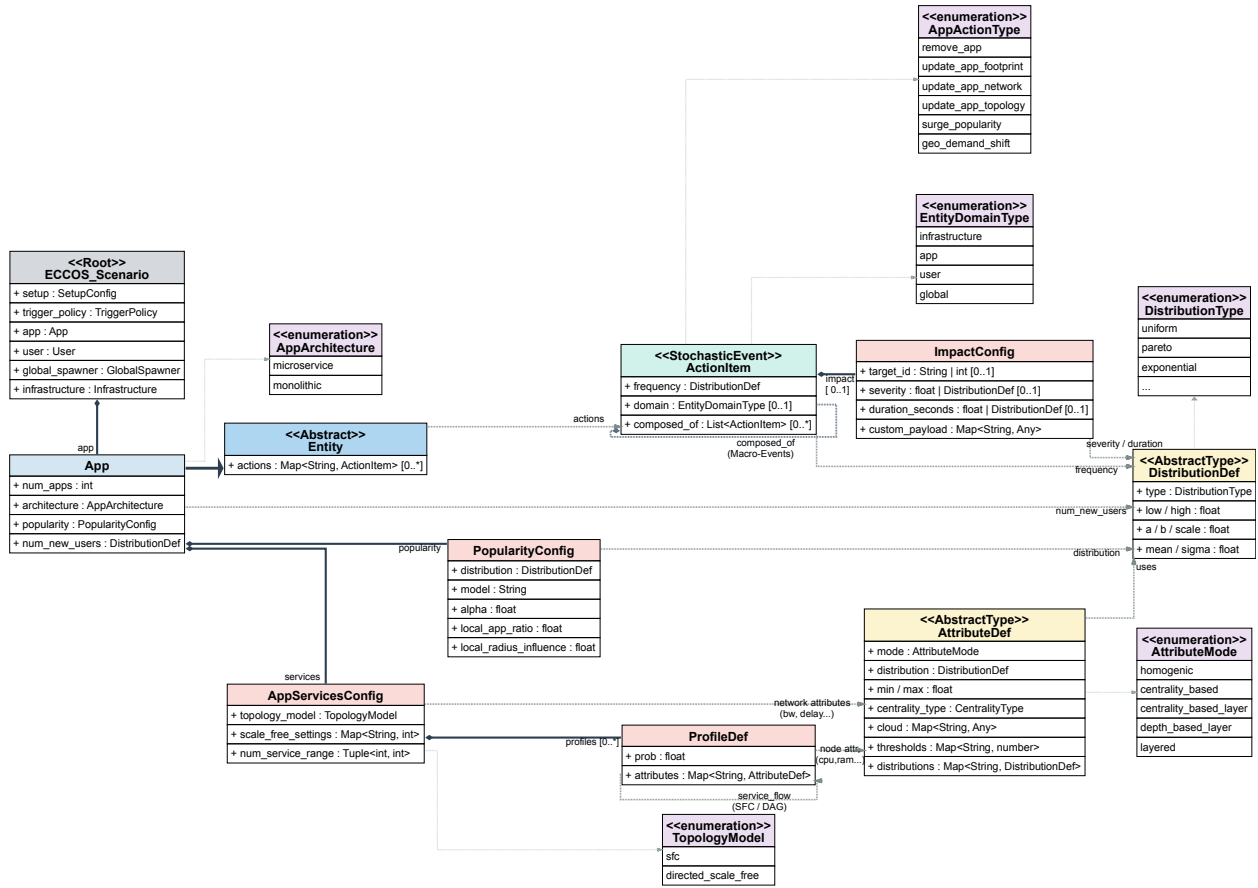


FIGURE 4 | UML model for application definition.

radius before signal degradation are stochastically sampled from predefined statistical distributions configured in the system [44]. Additionally, the movement patterns are determined by one of the two available mobility models: the Manhattan mobility model strictly constrains user trajectories to an orthogonal grid, mimicking the topology of city streets and avenues [45, 46]; the random direction model provides an unconstrained, isotropic random walk, better suited for simulating open-space environments such as parks, rural areas, or large event venues [47]. In both cases, the probability on changing the directions of previous movements/crossings is also determined in the YAML config file.

3.1.3 | Application Definition

Figure 4 presents the UML schema used to organize the application-level configuration elements considered in this subsection. Rather than enumerating every YAML field in the main text, the discussion concentrates on how ECCOS models application structure, resource demand, and popularity, with the detailed configuration parameters documented in the corresponding appendix (Section B.5).

In the context of evaluating service placement strategies in distributed edge environments, the computational workloads, the structural dependencies of applications, and the popularity patterns are influencing features and requires accurately modeling [11].

Regarding the structural dependencies of applications, the simulator considers each application as a set of inter-related services [48]. The simulator considers a fixed number for the initial number of applications, but the number of services composing each application is sampled from a uniform distribution within a configurable length. The user is able to decide whether applications are handled as a unique block, obtained by encapsulating all services in an indivisible entity (monolithic), or as independent services with different placement decisions (microservices) [49].

In the monolithic case, the simulator treats each application as a single self-contained computational entity whose attributes and features are calculated as the sum of the attributes of its component services. The placement strategy therefore

assigns the entire application to a single node. This mode is particularly relevant for simulating legacy systems, lightweight edge scripts, or highly coupled computing tasks [50].

In the microservice case, applications are treated as distributed sets of loosely coupled, fine-grained services with explicit dataflow dependencies. These dependencies can be configured either as a strict linear sequence, representing Service Function Chaining (SFC) [51], or as directed scale-free graph topologies (DAG) [48]. In this latter case, the simulator implements a directed scale-free network generation mechanism (typically leveraging preferential attachment algorithms similar to the Barabási–Albert model), dictating the initial network seeds (number of initial nodes) and the degree of connectivity for newly appended nodes. The resulting DAGs reflects the presence of central "hub" services and the intricate branching logic inherent in highly scalable, distributed cloud-native applications [52].

SFCs are representative of traditional telecommunication network service deployments, such as deep packet inspection or sequential stream processing pipelines, where data flows traverse a predetermined sequence of virtualized network functions [53]. On contrary, DAGs model the complex, non-linear dependencies characteristic of modern, web-scale microservice architectures [54].

Regarding computational workloads, applications are formally defined by a set of generalized resource-requirement profiles [11], such as IoT telemetry, video streaming... These profiles are randomly selected according to a categorical distribution parameterized by user-defined probabilities. Each profile is characterized by a dynamic set of multidimensional attributes (e.g., CPU, RAM, storage, and network bandwidth). These attributes represent the computational resource demands that services impose on infrastructure nodes. Similarly, communication resource consumption over network edges is modeled for the entire application through a dynamic set of attributes. In both cases, attribute values are initialized according to their underlying stochastic distributions. Attribute names must be mapped to the corresponding infrastructure attributes, whether node- or edge-level, in order to manage constraints such as resource capacity limits.

Applications are also configured according to their popularity, which is determined by a user-defined stochastic distribution. This popularity scheme can be expressed either as a general global request probability or as a geographically localized request probability [55]. This spatio-temporal popularity model regulates how users interact with services, allowing the framework to emulate realistic traffic-demand distributions for both globally popular and geographically localized applications. Consequently, applications are partitioned into global and local domains through a configurable percentage of the full number of applications.

The global applications are requested from any region with the same popularity-based probability. On contrary, local applications are assigned with a center of influence represented by a 2D coordinate within the simulation space [56]. The probability of requesting a local application (popularity) is dynamically adapted for new users created inside the local application influence radius, increasing the local application popularity according to the distance between the user and the application's center of influence [57]. In other words, the closer the user is to the center of a local application, the more likely the user is to decide to use it. Conversely, if the user is created beyond the influence radius, this probability boost is not considered, and the application request is governed only by its baseline global popularity as defined by the selected statistical distribution [58].

To guarantee that every newly deployed application is immediately subjected to a representative, randomized volume of user requests, the simulator creates a set of new active users for each new application. This allows the service placement algorithms to be evaluated under immediate operational demand.

3.2 | Ecco-Generator: Discrete-Event Stochastic Engine

The ECCO-GENERATOR is the core stochastic engine of the ECCOS framework. Unlike static placement simulators, ECCOS relies on a discrete-event simulation (DES) paradigm to model the extreme volatility and spatiotemporal fluctuations inherent to the Computing Continuum. This module is responsible for injecting entropy into the system across three main entities: Users (Section 3.2.2), Applications (Section 3.2.3), and Infrastructure (Section 3.2.1).

The simulator operates by generating a list of timestamped events, each one associated with a specific simulation time and with the instances generator (global spawner), or with an already generated instance of a system entity (user, application, node, or edge). These events are then processed in chronological order according to their scheduled simulation time. As a result, the simulation clock does not advance continuously, but discretely: it jumps from one event to the next, updating the system state only when an event is executed [59]. This event-driven mechanism enables ECCOS to efficiently capture dynamic behavior while preserving an explicit temporal ordering of all changes occurring in the simulated Computing Continuum [60].

Each event represents a state transition in the simulated environment and is ultimately reflected in the contextual information managed by ECCO-STATE (Section 3.3). For some event classes, the configuration must specify not only the event

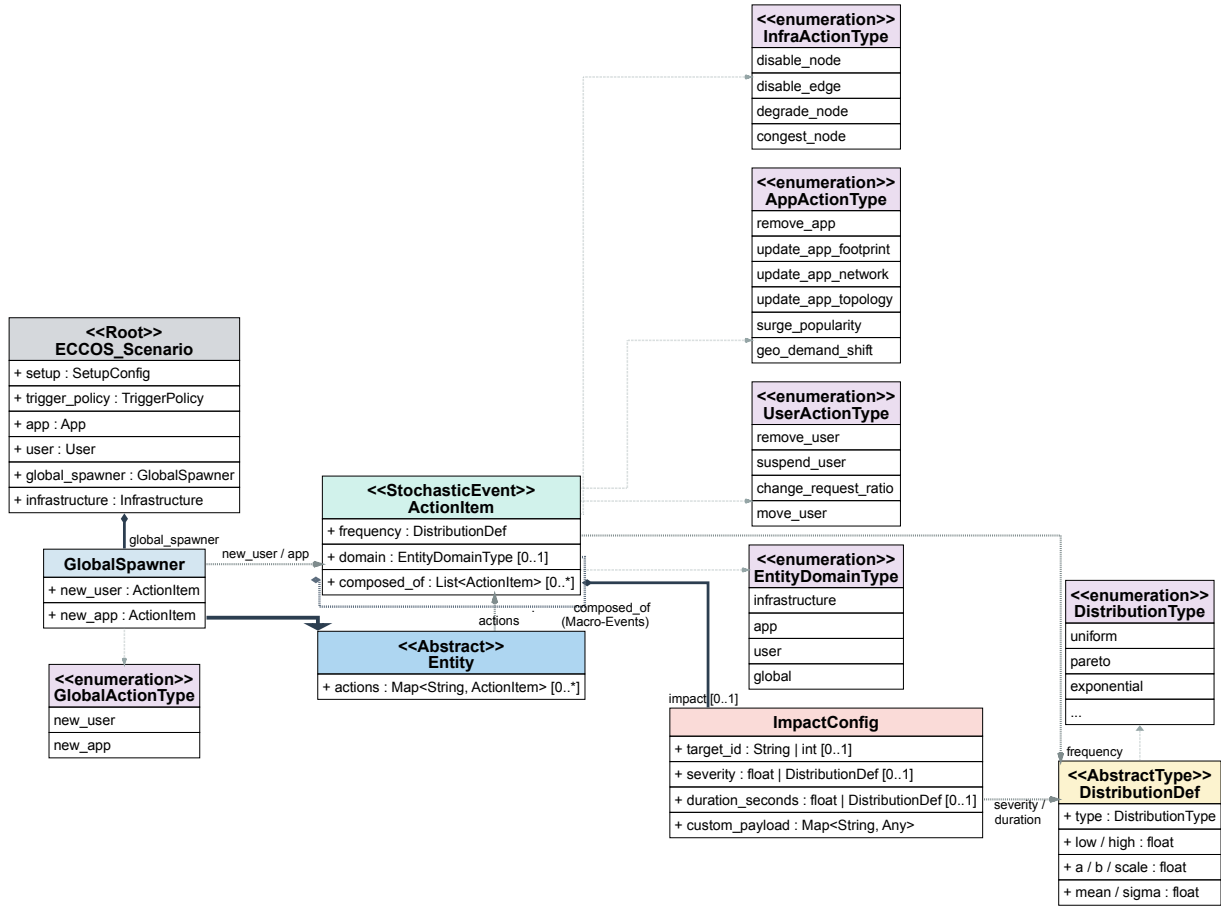


FIGURE 5 | UML model for the definition of base events and user-defined encapsulated events.

identifier and the statistical distribution (Section A) that determines when it is triggered (frequency), but also the operational parameters that define how the state transition is applied and its impact (impact) in terms of magnitude, target selection rules, affected resources, recovery behavior, or any additional effect to be required to execute the corresponding transition.

Dynamic simulation events are governed by an independent PRNG stream completely decoupled from the foundational infrastructure, application, and user generation phases (Section 3.1). By isolating event-level stochasticity into a dedicated, independently seeded generator, the simulation engine ensures exact execution-to-execution repeatability of dynamic traces across arbitrary time horizons [61]. This separation allows researchers to inject distinct stochastic event schedules or operational anomalies into identically initialized infrastructure and workload baselines. Consequently, observed variations in quality-of-service or placement algorithm performance can be unequivocally attributed to the evaluated policies rather than underlying stochastic drift.

In addition to the base events provided by the simulator, ECCOS also supports event encapsulation, allowing users to define composite events that combine multiple elementary state transitions into a higher-level simulation action (Section 3.2.5). Importantly, ECCOS does not require all event types to be enabled in a given experiment: events omitted from the YAML configuration are simply disabled. Together, event omission and event encapsulation increase experimental control: researchers can disable irrelevant sources of dynamism by not defining them in the YAML configuration, while also composing higher-level events from existing elementary transitions when more complex behaviors are required.

The general configuration model that captures how both basic simulator events and user-defined encapsulated events are specified is shown in Figure 5. This UML view provides a common reference for the following subsections, where the individual event families are described according to their target entity, frequency model, impact parameters, and optional composition mechanisms. Additionally, specific details of the YAML structure for events configuration are documented in several appendices (Sections B.2, B.4, B.6, B.7 and B.8,)

3.2.1 | Infrastructure Events

TABLE 1 | Stochastic action catalog and impact configuration schema for the infrastructure domain.

Action Specification		Impact Configuration Schema		
Action Name	Action Description	Impact Attribute	Attribute Description & Semantic Meaning	
disable_node	Completely shuts down a physical server, forcing service re-routing.	distribution_to_enable_node	Probability distribution governing downtime duration before auto-revival (revive_node).	
disable_edge	Severs a network link, triggering shortest-path recalculations.	distribution_to_enable_edge	Probability distribution governing link downtime duration before auto-restoration.	
degrade_node	Temporarily reduces available computational capacity on a physical server.	target_attributes	List of server resource metrics to degrade (e.g., ["cpu", "ram"]).	
		p_loss_dist	Probability distribution specifying the percentage reduction of server capacity.	
		distribution_to_restore_node	Distribution governing degradation duration before full capacity is restored.	
congest_edge	Degrades or inflates network link attributes due to congestion.	target_attributes_loss	Specific link attributes targeted for capacity loss.	
		target_attributes_multiplier	Specific link attributes targeted for delay inflation.	
		p_loss_dist	Distribution specifying bandwidth reduction or packet loss percentage.	
		multiplier_dist	Probability distribution specifying the latency scaling multiplier.	
		distribution_to_clear_edge	Distribution governing congestion duration before normal link metrics return.	

The physical layer of the Computing Continuum is highly volatile. The simulator incorporates events to model availability of the nodes and the edges of the infrastructure (Table 1). ECCOS distinguishes between total failures (blackouts) and temporal degradations (brownouts).

Total failures simulate power outages, hardware crashes, or cable cuts affecting individual infrastructure components [62]. ECCOS models these failures through instance-level events associated with specific entities of type node or edge, rather than as global failure processes over the whole infrastructure class. Accordingly, the edge (disable_edge) and node (disable_node) disable events are scheduled independently for each node or edge instance, and their frequency distributions describe the failure behavior of that specific component. When one of these instance-level events is triggered, the affected component is removed from the operational graph, its capabilities become unavailable, and any workloads depending on it must be evacuated or re-placed. The recovery process is also defined at the instance level: once the failure is generated, ECCOS schedules the corresponding restoration event using the impact distribution associated with that event, thereby modeling an individual Mean Time To Repair (MTTR) behavior for the failed node or edge.

In addition, degradation events are also implemented in the simulator. In Edge computing, partial degradations are far more common than total failures. ECCOS models computational throttling (e.g., thermal throttling, OS contention) via degrade node event (degrade_node) [63], and network interference via congest edge event (congest_edge) [64].

During a node degradation event, a node temporarily loses a percentage of its nominal capacity over the user-selected node attributes. Similarly, the network congestion mimicks network proprieties. These two events are paired with the recovery events, that restore the initial attributes values and ensure the infrastructure eventually returns to its exact nominal state. This allows to simulate, for example, situations such as a complex trade-off: whether to migrate away from a congested node at a high cost, or temporarily endure the QoS penalty until the brownout clears.

3.2.2 | User Events

TABLE 2 | Stochastic action catalog and impact configuration schema for the user domain.

Action Specification		Impact Configuration Schema	
Action Name	Action Description	Impact Attribute	Attribute Description & Semantic Meaning
remove_user	Permanently terminates a user session.	–	–
move_user	Displaces user coordinates (X, Y).	–	–
suspend_user	Temporarily halts user session and stops workload generation.	distribution_to_resume_user	Distribution governing suspension duration before auto-resumption.
change_request_ratio	Dynamically scales a user's workload generation frequency.	multiplier	Probability distribution generating the multiplier applied to the user's request ratio.

In Edge and 5G/6G environments, user mobility and connectivity volatility are the primary drivers for workload migrations. ECCO-GENERATOR models user behavior through continuous movement, request rate fluctuations, and temporary or permanent disconnections (Table 2) [65].

The most basic user-level behavior is permanent disconnection, represented by removing a user from the system (`remove_user`). This event only requires modeling its statistical occurrence and does not require any additional impact model. On contrary, temporary disconnections (`suspend_user`) requires of a more fine-grained configuration.

A user can lose connectivity either by moving outside the coverage radius, resulting in a Radio Link Failure [66], or stochastically through the suspend user event, which models temporary disconnections such as tunnel traversal, device hardware failures, sudden network outages, or IoT deep-sleep cycles [65]. In the latter case, the configuration must define both the statistical distribution governing how often a given user is temporarily disconnected and the impact distribution that determines the disconnection duration, that is, the time elapsed until the user is reconnected.

In the case of user mobility, the simulation framework decouples the intrinsic mobility properties of user entities from the stochastic triggers that govern their physical displacement, represented by the move user event (`move_user`). The temporal execution of spatial displacement is strictly driven by the move user event, whose occurrence frequency must be explicitly defined via stochastic distributions in the simulation configuration for ECCO-GENERATOR, enhancing the realism and variability of the spatial dynamics [39].

The mobility model, user speed, trajectory-change probabilities, and coverage radius are defined by ECCO-ARCHITECT (Section 3.1). Under the Manhattan mobility model, each move user event extends the user's previous trajectory by advancing along the orthogonal street grid. When the user reaches a street intersection, the direction-change probabilities are evaluated, and any remaining movement distance is applied along the selected outgoing direction [67].

Conversely, the Random Direction mobility model is not constrained by predefined paths. Each user preserves its previous direction vector, and every `move_user` event first evaluates the configured probability of changing direction [68]. If no change is triggered, the user continues along the same vector; otherwise, a new direction is sampled and the position is updated according to the user's speed.

Once the user's new position is computed by the mobility model, ECCO evaluates its proximity to the edge infrastructure. The user is dynamically associated with the nearest edge node when it remains within the configured coverage radius; otherwise, a signal-loss condition is triggered and the user's connectivity state is temporarily suspended [69]. Consequently, not every move user event produces a roaming or disconnection action. A roaming action only occurs when the nearest edge node changes due to the user's new geographical position, whereas a disconnection only occurs when the user moves outside the coverage range of all available edge connection nodes [70]. Users disconnected as a result of mobility remain disconnected until subsequent movement events bring them back within the coverage radius of an edge node.

Finally, the browsing behaviour of the users can be also modified during the simulation by altering the request ratio for the application that the user requests (`change_request_ratio`), emulating the highly dynamic nature of real-world mobile traffic [71]. The change request ratio is governed by its inter-arrival time distribution. The event incorporates a directional probability threshold that model the anomaly as a traffic burst (multiplying the current request ratio by a sampled scalar) or a traffic valley (dividing it by the same scalar). The event behaves as a bounded, self-restoring state machine. Each triggered fluctuation dynamically schedules a secondary restoration event, parameterized by its own time distribution. Furthermore, the architecture guarantees state consistency under edge cases; if a user suffers a physical disconnection or hardware failure during an active burst, the system sanitizes the variables to ensure that the user resumes normal traffic behavior upon reconnection, preserving the integrity of the simulation.

3.2.3 | Application Events

TABLE 3 | Stochastic action catalog and impact configuration schema for the application domain.

Action Specification		Impact Configuration Schema	
Action Name	Action Description	Impact Attribute	Attribute Description & Semantic Meaning
remove_app	Decommissions an application graph.	–	–
update_app_footprint	Scales computational demands (CPU, RAM...) of microservices.	multiplier	Distribution scaling computational resource requirements of service nodes.
		sigma_footprint	Noise distribution/standard deviation introduced during footprint resizing.
update_app_network	Alters network links demands generated across service intercommunication edges.	multiplier	Distribution scaling network requirement demands.
		sigma_net	Noise distribution introduced into service link network constraints.
update_app_topology	Mutates service graph structure by adding or removing microservices.	l_min	Distribution specifying the lower bound for microservice count in the DAG/SFC.
		l_max_global	Distribution specifying the upper bound for microservice count in the DAG/SFC.
surge_popularity	Triggers a transient upward surge in application request volume.	transient_prob	Probability distribution determining whether the spike is transient or permanent.
		duration_dist	Distribution governing the duration of a transient surge before returning to normal.
drop_popularity	Induces a transient decrease in application popularity and request load.	transient_prob	Probability distribution determining whether the drop is transient or permanent.
		duration_dist	Distribution governing the duration of a transient popularity drop.
geo_demand_shift	Relocates geographic center of gravity of application demand.	jump_prob	Probability distribution determining abrupt geographic jump vs. smooth drift.
		drift_dist	Spatial displacement distribution ($\Delta X, \Delta Y$) applied during smooth drifts.

Modern applications deployed in the Continuum are not monolithic nor static; they rely on microservices composition and experience geographical and temporal demand shifts [72]. Beyond simple application deletion (`remove_app`), applications may undergo lifecycle updates that affect their resource demands or service composition (Table 3) [73]. Day-two operations can significantly alter an application's footprint. The simulator provides three mechanisms for modeling application lifecycles: changes in computational workload (`update_app_footprint`), changes in network demand (`update_app_network`), and changes in service composition (`update_app_topology`).

The events that simulate changes in application workloads and network usage operate in a similar manner. They respectively modify the resource requirements of constituent microservices and the application's network requirements. Their stochastic impact is parameterized by a user-defined probability distribution that represents the standard deviation of the applied noise. This perturbation either proportionally scales the application's computational resource demands during its lifecycle or simulates sudden network demand spikes and varying Quality of Service (QoS) conditions [74].

The event that changes an application's service composition is applicable only to microservice-based architectures. It simulates the dynamic evolution of applications through the stochastic insertion or deletion of microservices, while enforcing lower and upper bounds on the number of services per application [74].

ECCOS simulates viral trends or "Flash Crowds" through events that abruptly modify an application's popularity, either by increasing it through a surge (`surge_popularity`) or decreasing it through a drop (`drop_popularity`) [75]. This modification only affects the application-selection process of users created after the event, thereby increasing or decreasing the number of new users likely to request the application. Existing application popularity values and the request ratios of users already associated with these applications remain unchanged, and they are altered only by user's events. The phenomenon can be transient or permanent. When it is transient, as in the case of a live sporting event, a paired restore-popularity event is scheduled after a specified period of time [75].

Finally, and for the cases of localized applications, defined as subsets of applications with higher popularity in hot-spot areas, the emulator allows to simulate the spatial migration of user interest (`geo_demand_shift`). This is achieved by displacing the hot spot either through random jumps to new coordinates or through small incremental movements from its current location. As a result, the optimization engine may need to perform live migrations of edge microservices to track the evolving user base [76].

3.2.4 | Global Spawner Events

TABLE 4 | Stochastic action catalog and impact configuration schema for the `global_spawner` domain.

Action Specification		Impact Configuration Schema	
Action Name	Action Description	Impact Attribute	Attribute Description & Semantic Meaning
<code>new_user</code>	Dynamically instantiates a new user from scratch (<i>ex novo</i>).	–	–
<code>new_app</code>	Dynamically instantiates a new application DAG/SFC from scratch.	<code>num_new_users</code>	Distribution determining the number of initial users spawned for the new app.

Creating new users (`new_user`) and applications (`new_app`) is a global simulation event that represents the external arrival of a new entity into the scenario. The global spawner is the ECCO-GENERATOR component responsible for introducing these new simulation entities (Table 4) [77]. Its role differs from actions attached to existing instances: user or application actions modify, move, remove, or update specific objects, whereas creation events add entirely new objects to the simulation state. This distinction is necessary because the internal event-generation logic assigns configured actions individually to each existing entity. For example, if a remove-user event is configured, the simulator schedules an independent removal countdown for User 1, another for User 2, and so on. Such behavior is appropriate for instance-specific events, but not for entity creation, which must instead be defined in a general event container such as the global spawner [78].

This design choice preserves the intended DES semantics: one global arrival process controls the birth of new entities, while per-object action blocks control the lifecycle and behavior of entities after they have been instantiated. This design cleanly separates exogenous simulation dynamics from endogenous entity behavior [78].

3.2.5 | User Composition of Encapsulated Events

A significant challenge in discrete-event simulation for edge and cloud environments is capturing the complex, cascading nature of real-world phenomena [79]. While traditional simulators treat events as isolated occurrences affecting a single entity (e.g., a node failure, a user arrival), real-world occurrences often trigger a chain of heterogeneous impacts across the infrastructure. For instance, a catastrophic power outage not only disables a physical node but simultaneously terminates the applications hosted within it and disconnects the locally attached users. Furthermore, macroscopic events such as localized flash crowds require coordinated actions across multiple entities.

ECCOS simulators allows user-defined encapsulated events, which acts as a logical container that defines a single probability distribution for its occurrence frequency, but its execution impact is decomposed into a collection of primitive sub-actions affecting heterogeneous entities [80].

This is achieved with a declarative encapsulated events architecture, and defining the use of the `composed_of` directive within the event's YAML configuration. When the simulator's event engine dispatches the macro-event, it iterates through the `composed_of` list, dynamically resolving dependencies and executing the underlying base actions within the same simulation time step.

The simulator's compositional event architecture is designed to be completely orthogonal to the origin of the trigger. Consequently, composite events can be associated both with concrete domain instances or with the global spawner to orchestrate multi-step structural dynamics.

When an encapsulated event involves multiple entities, identifying the specific instance over which the sub-action must be applied is a critical challenge. To solve this, the simulator implements a dynamic target resolution engine. This engine provides granular control over how target instances are dynamically resolved during the simulation loop. The resolution engine offers several configuration alternatives to identify the affected instances: the same instance that generates

the macro-event; a uniform random selection from all the instances; an specific instance identified by its identifier; a delegated and self-resolution of the simulator; or to a group of instances.

In the latter case, when the dynamic target-resolution engine is configured to select a group of instances, the event affects a cluster of N elements simultaneously [81]. The cluster size can be defined either as an absolute integer (e.g., exactly 5 nodes) or dynamically as a probability or percentage (e.g., 0.15 to affect 15% of the currently active users in the simulator). The target resolution of this group requires additional parametrization, supporting explicit list selection [82], uniform random sampling [81], a hybrid approach that selects the originating instance plus $N - 1$ random instances [83], or a proximity-based selection model [84]. This last alternative considers two cases: selecting a random anchor element and its proximate elements, or selecting the *self*-generating element and its proximate elements.

This contextual proximity model is particularly useful for modeling localized phenomena. Because proximity depends on the entity type, the resolution engine implements heterogeneous proximity metrics: for infrastructure nodes, proximity is defined as the topological shortest-path distance within the network graph [84]; for users, it corresponds to the topological distance between the access points to which they are actively connected [85]; and for applications, it is based either on geographical affinity, when applications share the same spatial hot-spot mapping, or on popularity adjacency, grouping applications with similar popularity ranks [86].

3.3 | Ecco-State: System State Manager

The ECCO-STATE module constitutes the state persistence and analytical accounting core of the simulator, acting as the authoritative *Single Source of Truth* regarding the instantaneous physical and logical status of the underlying infrastructure at any discrete time step t . Its primary architectural responsibility is to enforce mathematical consistency and structural invariants across the discrete-event stochastic transitions driven by ECCO-GENERATOR and the placement decisions computed by ECCO-SOLVER.

At the operational level, ECCO-STATE maintains the unified structural graph of the tiered physical infrastructure $\mathcal{G}$, comprising heterogeneous compute nodes $\mathcal{N}$ and interconnecting network links $\mathcal{L}$, alongside the Directed Acyclic Graphs (DAGs) representing applications $\mathcal{A}$ and the active user population $\mathcal{U}$. Each time an event from the event list generated by ECCO-GENERATOR is processed, the simulator state changes and all these structures and state variables must be updated consistently [87], handled by ECCO-STATE.

To guarantee physical feasibility throughout the simulation horizon, the module also implements a rigorous residual-capacity accounting engine. A fundamental architectural strength of ECCO-STATE is its dynamic, schema-agnostic resource accounting model. Unlike domain-specific edge simulators that hardcode a rigid tuple of physical capacities (typically limited to CPU cores, RAM memory, and link bandwidth), ECCO-STATE operates over arbitrary, user-defined multi-dimensional attribute spaces defined via configuration manifests [88].

By employing dynamic introspection over graph node and edge dictionaries, the state manager natively tracks any set of custom quantitative attributes (features) $F = \{f_1, f_2, \dots, f_k\}$ specified by the user (e.g., hardware accelerators such as GPUs/TPUs, disk storage constraints, or secure enclave quotas) [89]. During state validation and solver formulation, the system iterates over all numeric capacities present in the infrastructure topology and automatically enforces residual capacity invariants across matching microservice demands:

$$\sum_{m \in \mathcal{M}_n} d_{a,m}^f \leq C_n^f, \quad \forall n \in \mathcal{N}, \forall f \in F \quad (1)$$

where $d_{a,m}^f$ represents the demand of microservice m for attribute f , C_n^f denotes the physical capacity of node n for attribute f , and $\mathcal{M}_n$ represents the set of microservices from any application that are currently placed in node $n \in \mathcal{N}$ and determined by the current placement dictionary $\mathcal{P}^{(t)}$. If an attribute is omitted from either the infrastructure or application specification, the accounting engine safely evaluates demand to zero without schema violations, ensuring complete backward compatibility and high extensibility [90].

In summary, every structural state transition, whether triggered by external stochastic events (e.g., user arrivals, mobility, or infrastructure degradation) or internal solver allocations, is transactionally validated by ECCO-STATE to prevent resource oversubscription [89]. Furthermore, the module constructs immutable, deterministic state snapshots ($S^{(t)}$) that are exposed via standardized interfaces to optimization engines.

Finally, a distinguishing architectural capability of ECCO-STATE is its support for *latent or non-linear state transitions* within application and user lifecycles, overcoming the limitations of traditional binary simulators that restrict services to mutually exclusive *active* or *terminated* states.

Rather than naïvely destroying deployed microservices and releasing allocated memory upon a temporary user disconnection, ECCO-STATE transitions the affected session into a *latent soft-state retention mode*. In those cases, the effective

request generation rate of the disconnected user drops to zero, immediately relieving network consumptions but retaining the node resources allocation, emulating static reservation and dynamic load over the resource attributes [91].

3.4 | Ecco-Solver: Decoupled Optimization Engine

A cornerstone of the ECCOS simulation framework is its strict architectural separation between discrete-event simulation dynamics and mathematical optimization [92]. The ECCO-SOLVER module acts as a completely decoupled optimization engine responsible for computing the spatial mapping of application microservices across the underlying physical infrastructure. By isolating the placement logic behind a uniform abstraction layer, ECCOS allows researchers to design, prototype, and benchmark custom placement algorithms, ranging from exact Integer Linear Programming (ILP) formulations to fast heuristics or Reinforcement Learning policies, without modifying the core event scheduler or system state tracking.

Every solver in ECCOS inherits from the abstract base class `BaseSolver`, which establishes a stateless, standardized interface for decision-making. When a placement policy is triggered at simulation time t , the simulation runner invokes the solver through a single contract:

$$(\mathcal{P}^{(t)}, \mathcal{C}^{(t)}) \leftarrow \text{solve}(\mathcal{G}, \mathcal{A}, \mathcal{U}, \Omega_{\text{cfg}}, \mathcal{P}^{(t-\Delta t)}) \quad (2)$$

To implement a new custom solver (such as an exact mathematical model or a fast greedy heuristic), a developer must consider the five structured inputs provided by the simulator and the required output specification [93]:

1. **Infrastructure Graph ($\mathcal{G}$):** A directed graph representing the active physical infrastructure. Nodes $n \in \mathcal{N}$ expose multi-dimensional resource capacities C_n^f for each hardware attribute $f \in F$, such as $f \in \{\text{CPU, RAM, Storage}\}$, along with binary operational states. Edges representing network links provide all-pairs shortest-path propagation delays $D(i, j)$ for any two physical nodes $i, j \in \mathcal{N}$ and, similarly to the nodes, their multi-dimensional resource attributes, such as $f \in \{\text{Delay, Bandwidth}\}$.
2. **Application Workload ($\mathcal{A}$):** The active set of Service Function Chaining (SFC) or Directed Acyclic Graph (DAG) applications. Each application $a \in \mathcal{A}$ consists of a set of ordered microservices $m \in \mathcal{M}_a$ with computational demands $d_{a,m}^f$, connected by directed virtual links that models the microservice's interactions in application a $\mathcal{I}_a = \{(m_1, m_2)\}$.
3. **User Population ($\mathcal{U}$):** The active mobile user instances generating traffic requests. Each user $u \in \mathcal{U}$ is characterized by their requested application $a(u) \in \mathcal{A}$, their currently associated network access point $n(u) \in \mathcal{N}$, and their dynamic request ratio multiplier λ_u .
4. **Execution Configuration (Ω_{cfg}):** Runtime hyperparameters loaded from the solver YAML configuration file. This parameters are solver-specific, and different parameters would be consider for different solvers. For example, for ILP solvers, the list of parameters could include optimality tolerances (`gapRel`), execution timeouts (`timeLimit`), and objective weighting coefficients.
5. **Previous System State ($\mathcal{P}^{(t-\Delta t)}$):** A structured snapshot of the placement dictionary at the preceding decision epoch. Providing the historical placement is critical for stateful optimization, enabling solvers to evaluate service migration costs and prevent hysteresis or route flapping.

Regardless of internal algorithmic complexity, every solver must return a standardized tuple $(\mathcal{P}, \mathcal{C})$. To accommodate both single-instance and multi-replica distributed deployments across the Computing Continuum [93], the placement mapping $\mathcal{P}$ is formulated as a nested dictionary assigning each microservice unique identifier $m \in \mathcal{M}_a$ of application $a \in \mathcal{A}$ to a non-empty subset of physical hosting nodes $\mathcal{N}_{a,m} \subseteq \mathcal{N}$:

$$\mathcal{P} = \{a : \{m : \mathcal{N}_{a,m}, \forall m \in \mathcal{M}_a\}, \forall a \in \mathcal{A}\} \quad (3)$$

In standard non-replicated deployments, the hosting set contains a single target node ($|\mathcal{N}_{a,m}| = 1$), whereas in multi-instance load-balancing or redundant workloads, $\mathcal{N}_{a,m}$ contains multiple simultaneous host nodes ($|\mathcal{N}_{a,m}| \geq 1$). The scalar $\mathcal{C} \in \mathbb{R}^+$ represents the evaluated objective cost of the resulting placement mapping, or returns a heavy penalty ($\mathcal{C} = \text{infeasible_penalty}$) if the problem is mathematically infeasible due to physical capacity, or other, constraints.

In a high-frequency Discrete-Event Simulation where hundreds of stochastic events occur per simulated minute, invoking an NP-hard mathematical solver on every single event would cause computational bottlenecks [94]. To ensure simulation scalability, ECCOS incorporates a dedicated trigger policy manager that decouples event arrival frequency from solver execution frequency or conditions [95].

Configurable via the solver YAML specification attribute `trigger_policy.type` (Section C), the trigger manager evaluates every incoming simulation event against a user-defined policy to decide whether to trigger solver recalculation:

- **Every Event** (`solve_all`): Triggers the solver after each single event.
- **Solver Disable** (`solve_none`): The solver is not executed after any event. The output only includes the list of events in the system without the placement solutions.
- **Periodic Time Intervals** (`solve_every_t_seconds`): Invokes the solver strictly at fixed simulation time periods `interval_seconds`.
- **Event Batching** (`solve_every_n_events`): Triggers recalculation after every `batch_size` cumulative discrete events across the ecosystem.
- **User-listed Event Interrupts** (`solve_on_event_types`): An event-driven policy that executes the solver only upon the arrival of a limited list of events in attribute `critical_events`.
- **Temporal Windows** (`solve_time_windows`): Confines optimization execution to specific lists (windows) of simulation time slots $[t_{\text{start}}, t_{\text{end}}]$.
- **Event Index Ranges** (`solve_time_windows`): Confines optimization execution to specific lists (windows) of events intervals $[i_{\text{start}}, i_{\text{end}}]$.
- **Random Event Selection** (`solve_random_prob`): Triggers solver execution probabilistically for each event according to the specified probability value.
- **Pattern-based Selection** (`solve_custom_pattern`): A sequence of Boolean values manually specifies the events for which the solver must be triggered (pattern). This pattern is applied cyclically throughout the simulation.
- **Hybrid Policy** (`combined`): Combines periodic time-driven recalculations every `interval_seconds` seconds (`solve_every_t_seconds`), with immediate event-driven interrupts whenever a user-listed event in `critical_events` (`solve_on_event_types`).

Note that `type:solve_none` uses only the event-trace generation capabilities of ECCOS, but it opens the possibility of employing the framework as a discrete-event generator that can be integrated with third-party applications lacking an equally powerful mechanism for defining and producing discrete events.

ECCOS currently provides three built-in solvers: two ILP-based formulations (Sections 3.4.1 and 3.4.2) and one greedy heuristic (Section 3.4.3). The solver used in a given simulation is selected in the solver YAML configuration file through the `setup.solver` attribute. In any case, as discussed above, users can extend the set of available solvers by implementing the `BaseSolver` interface, and then make the new solver visible and selectable from the YAML configuration by registering it through the `SolverFactory` class.

3.4.1 | Single-Objective ILP: Pure Latency Optimization

The first baseline formulation implemented in ECCOS (`ILPSingleObjectiveSolver`) models the classic latency-minimal service placement problem. Its sole objective is to minimize the total weighted end-to-end network delay perceived by active users, accounting for both ingress access latency and internal microservice-to-microservice communication delay [96].

Decision Variables. The formulation defines two sets of decision variables:

$$x_{a,m,n} \in \{0, 1\}, \quad \forall a \in \mathcal{A}, m \in \mathcal{M}_a, n \in \mathcal{N} \quad (4)$$

$$y_{a,i,n_1,n_2} \in [0, 1], \quad \forall a \in \mathcal{A}, i = (m_1, m_2) \in \mathcal{I}_a, n_1, n_2 \in \mathcal{N} \quad (5)$$

where $x_{a,m,n} = 1$ if microservice m of application a is hosted on physical server n . The continuous variable y_{a,e,n_1,n_2} represents the linearized link placement, taking value 1 if virtual interaction edge $i = (m_1, m_2)$ is mapped across the physical path from n_1 to n_2 [97].

Objective Function. Let $m_0^{(a)}$ denote the ingress (first) microservice of application a , and let $\Lambda_a = \sum_{u \in \mathcal{U}: a(u)=a} \lambda_u$ represent the aggregated request load for application a [96]. The objective function minimizes the weighted average latency by dividing the total latency by the sum of all request ratios:

$$\min \quad \frac{1}{\sum_{u \in \mathcal{U}} \lambda_u} \left(\sum_{u \in \mathcal{U}} \lambda_u \sum_{n \in \mathcal{N}} D(n(u), n) \cdot x_{a(u), m_0^{(a(u))}, n} + \sum_{a \in \mathcal{A}} \Lambda_a \sum_{i=(m_1, m_2) \in \mathcal{I}_a} \sum_{n_1 \in \mathcal{N}} \sum_{n_2 \in \mathcal{N}} D(n_1, n_2) \cdot y_{a,i,n_1,n_2} \right) \quad (6)$$

Constraints. The optimization is subject to exact placement, physical capacity, and flow linearization constraints [97]:

$$\sum_{n \in \mathcal{N}} x_{a,m,n} = 1, \quad \forall a \in \mathcal{A}, m \in \mathcal{M}_a \quad (7)$$

$$\sum_{a \in \mathcal{A}} \sum_{m \in \mathcal{M}_a} d_{a,m}^k \cdot x_{a,m,n} \leq C_n^k, \quad \forall n \in \mathcal{N}, k \in \{\text{CPU}, \text{RAM}\} \quad (8)$$

$$\sum_{n_2 \in \mathcal{N}} y_{a,i,n_1,n_2} = x_{a,m_1,n_1}, \quad \forall a \in \mathcal{A}, i = (m_1, m_2), n_1 \in \mathcal{N} \quad (9)$$

$$\sum_{n_1 \in \mathcal{N}} y_{a,i,n_1,n_2} = x_{a,m_2,n_2}, \quad \forall a \in \mathcal{A}, i = (m_1, m_2), n_2 \in \mathcal{N} \quad (10)$$

Constraint (7) guarantees that every microservice instance is deployed on exactly one operational server. Constraint (8) ensures that aggregate resource consumption does not exceed node capacity. Constraints (9) and (10) enforce topological flow conservation between microservice placement variables and virtual link variables. These two flow linearization constraints (9) and (10) are required to avoid quadratic non-linearities arising from the multiplication of binary placement variables, and the formulation introduces auxiliary link-mapping variables $y_{a,i,n_1,n_2} \in 0, 1$. Specifically, these constraints enforce that an application edge $i = (m_1, m_2)$ is mapped over physical path (n_1, n_2) if and only if microservices m_1 and m_2 are hosted on servers n_1 and n_2 , respectively. This guarantees exact accounting of inter-service network delays while preserving a computationally tractable Mixed-Integer Linear Programming (MILP) formulation.

3.4.2 | Multi-Objective ILP: Latency, Migration, and Consolidation

While pure latency optimization achieves optimal quality of service, applying it independently at each time step often leads to operational instability: minor shifts in user mobility can trigger massive microservice relocations across servers. To address stateful trade-offs, ECCOS implements a second solver (ILPMultiObjectiveSolver) that balances end-to-end latency against service migration overhead and server consolidation (energy efficiency).

Decision Variables. In addition to $x_{a,m,n}$ and y_{a,i,n_1,n_2} , the multi-objective model introduces a binary server activation variable:

$$z_n \in \{0, 1\}, \quad \forall n \in \mathcal{N} \quad (11)$$

where $z_n = 1$ if server node n hosts at least one active microservice instance.

Multi-Objective Cost Formulation. Let $\mathcal{P}^{(t-\Delta t)}(a, m) \in \mathcal{N}$ denote the physical server that hosted microservice m of application a in the previous simulation epoch. The migration penalty indicator is formulated as $(1 - x_{a,m,\mathcal{P}^{(t-\Delta t)}(a,m)})$, which evaluates to 1 whenever a microservice is moved to a different node.

The composite scalar objective function minimizes a weighted linear combination of latency, migration frequency, and active server footprint:

$$\min \quad w_{\text{lat}} \cdot \Phi_{\text{lat}} + w_{\text{mig}} \cdot \Phi_{\text{mig}} + w_{\text{srv}} \cdot \Phi_{\text{srv}} \quad (12)$$

where the individual cost components are defined as:

$$\Phi_{\text{lat}} = \sum_{u \in \mathcal{U}} \lambda_u \sum_{n \in \mathcal{N}} D(u(n), n) x_{a,m_0,n} + \sum_{a,i,n_1,n_2} \Lambda_a D(n_1, n_2) y_{a,i,n_1,n_2} \quad (13)$$

$$\Phi_{\text{mig}} = \sum_{a \in \mathcal{A}} \sum_{m \in \mathcal{M}_a} \mathbb{D}_{\text{active}}(a, m) \cdot (1 - x_{a,m,\mathcal{P}^{(t-\Delta t)}(a,m)}) \quad (14)$$

$$\Phi_{\text{srv}} = \sum_{n \in \mathcal{N}} z_n \quad (15)$$

where the active deployment indicator function $\mathbb{D}_{\text{active}}(a, m) \in 0, 1$ is evaluated to 1 if microservice $m \in \mathcal{M}_a$ was actively running in the preceding time step $(t - \Delta t)$, ensuring that newly instantiated microservices do not trigger false migration penalties. The non-negative weights w_{lat} , w_{mig} , and w_{srv} are configurable via the solver configuration YAML file, allowing users to tune the simulator between aggressive low-latency placement and highly stable, consolidated deployments.

Additional Constraints. In addition to constraints (7)–(10), the multi-objective solver couples microservice assignments to server activation status via a Big- M constraint [98]:

$$\sum_{a \in \mathcal{A}} \sum_{m \in \mathcal{M}_a} x_{a,m,n} \leq M \cdot z_n, \quad \forall n \in \mathcal{N} \quad (16)$$

where $M = \sum_{a \in \mathcal{A}} |\mathcal{M}_a|$ represents an upper bound on the total number of deployable microservices across the entire ecosystem.

3.4.3 | Demand-Aware Greedy Heuristic: Sub-Millisecond Attraction Placement

Exact Integer Linear Programming solvers guarantee global or Pareto-optimal placements but suffer from exponential worst-case computational complexity ($O(2^{|\mathcal{A}||\mathcal{M}||\mathcal{N}|})$). As infrastructure topologies scale to hundreds of Edge and Fog nodes, solving ILPs at every decision epoch can introduce undesirable simulation bottlenecks. To provide an ultra-fast baseline suitable for large-scale real-time orchestration, ECCOS includes a deterministic heuristic solver (*GreedySolver*) based on a demand-aware topological attraction strategy.

Application Prioritization. Rather than jointly optimizing all applications simultaneously, the greedy solver decomposes the placement problem sequentially by prioritizing applications with the highest user demand. Let $\mathcal{U}_a = \{u \in \mathcal{U} : a(u) = a\}$ denote the subset of active users requesting application $a \in \mathcal{A}$. The aggregate request intensity Λ_a for application a is formulated as:

$$\Lambda_a = \sum_{u \in \mathcal{U}_a} \lambda_u \quad (17)$$

The solver sorts the application set $\mathcal{A}$ into an ordered sequence $\vec{\mathcal{A}} = (a_{(1)}, a_{(2)}, \dots, a_{(|\mathcal{A}|)})$ such that $\Lambda_{a_{(1)}} \geq \Lambda_{a_{(2)}} \geq \dots \geq \Lambda_{a_{(|\mathcal{A}|)}}$, breaking ties deterministically by unique application identifier.

Topological Attraction Scoring. For each prioritized application $a \in \vec{\mathcal{A}}$, the heuristic evaluates all operational server nodes $n \in \mathcal{N}$ by computing a topological attraction score $\Psi(a, n)$. This metric quantifies the cumulative weighted shortest-path access latency from all users currently requesting application a to candidate server n :

$$\Psi(a, n) = \sum_{u \in \mathcal{U}_a} \lambda_u \cdot D(u_{\text{node}}, n), \quad \forall n \in \mathcal{N} \quad (18)$$

Candidate physical servers are ranked in ascending order of $\Psi(a, n)$, producing a prioritized node list where servers topologically closest to the application's active user centroid appear first.

Monolithic-First Allocation Strategy. Let $R_n^k(t)$ denote the remaining available capacity of resource $k \in \{\text{CPU}, \text{RAM}\}$ on physical node n at the current step of the greedy allocation loop. For each application a , the placement mapping is constructed via a two-stage allocation mechanism:

1. **Monolithic Co-location Attempt:** The solver first scans the ranked candidate nodes to find the best server n^* capable of co-locating the entire microservice chain $\mathcal{M}_a$ locally, satisfying:

$$\sum_{m \in \mathcal{M}_a} d_{a,m}^k \leq R_{n^*}^k(t), \quad \forall k \in \{\text{CPU}, \text{RAM}\} \quad (19)$$

If such a node exists, all microservices $m \in \mathcal{M}_a$ are assigned to n^* ($\mathcal{P}_{a,m} \leftarrow n^*$), eliminating inter-microservice network delay ($D(n^*, n^*) = 0$). Remaining node capacities are immediately decremented: $R_{n^*}^k(t) \leftarrow R_{n^*}^k(t) - \sum_m d_{a,m}^k$.

2. **Distributed Fallback Allocation:** If no single server possesses sufficient residual resources to host the entire application, the solver iterates through each microservice $m \in \mathcal{M}_a$ sequentially and deploys it on the first candidate server $n \in \mathcal{N}$ in the attraction-ranked list satisfying individual microservice demand ($d_{a,m}^k \leq R_n^k(t)$).

Feasibility and Output Evaluation. If any microservice $m \in \mathcal{M}_a$ cannot be allocated due to infrastructure resource exhaustion, the algorithm terminates immediately and returns an infeasible status tuple ($\emptyset$, infeasible_penalty). Otherwise, upon successfully placing all applications, the solver evaluates the standardized objective cost $\mathcal{C} \in \mathbb{R}^+$ across the finalized mapping $\mathcal{P}$ using the identical end-to-end weighted latency function applied by the single-objective ILP model, ensuring fair cross-algorithm benchmarking.

3.5 | Ecco-Bridge: Data Ecosystem and ML-Interface Engine

ECCOS exports highly structured, self-contained trace records. Rather than dumping unstructured log streams, the simulation engine formalizes every orchestration step into an independent, well-typed JSON snapshot designed to directly support data-driven modeling adapted to Machine Learning (ML) and Reinforcement Learning (RL) post-processing. While tabular formats such as CSV or streaming formats such as JSON Lines are standard for flat, purely numerical datasets, they prove fundamentally inadequate for representing the topological and hierarchical complexity of the Computing Continuum. A Computing Continuum state inherently comprises a non-Euclidean graph topology consisting of heterogeneous physical nodes, dynamic network edges with fluctuating inter-node propagation delays, and multi-layered service dependencies (users connected to applications composed of microservices deployed on hardware nodes). Flattening such nested graph structures into single-line CSV rows or JSON Lines strings inevitably destroys spatial adjacency and forces artificial serialization choices [99]. On contrary, hierarchical JSON provides an intuitive, human-readable representation of graph attributes (node_information, edges, user-to-node connectivity, and application mappings). Modern ML

pipelines, specifically GNN frameworks such as *PyTorch Geometric* or *DGL*, consume structured JSON records natively, converting each file directly into structured graph tensors without loss of relational fidelity.

Each simulation run generates a structured trace repository containing individual JSON snapshot files for each simulation step, i.e., for each event processing. An important design principle of our output architecture is that each step file is fully self-contained [100]. It encapsulates the physical pre-decision state of the Computing Continuum ($\mathcal{G}^{(t-\Delta t)} \cup \mathcal{A}^{(t-\Delta t)} \cup \mathcal{U}^{(t-\Delta t)} \cup \mathcal{P}^{(t-\Delta t)}$), the quantitative optimization metrics achieved by the solver at the preceding decision epoch ($\mathcal{O}^{(t-\Delta t)}$), the exogenous environmental event that triggers the simulation step ($\mathcal{E}^{(t)}$), the explicit placement differential or migration plan obtained from the difference between the previous placement and the new placement decision obtained by the solver in the current epoch ($\Delta\mathcal{P}^{(t)} = \mathcal{P}^{(t)} \ominus \mathcal{P}^{(t-\Delta t)}$), the resulting post-decision state ($\mathcal{G}^{(t)} \cup \mathcal{A}^{(t)} \cup \mathcal{U}^{(t)} \cup \mathcal{P}^{(t)}$), and the new quantitative optimization metrics achieved by the solver ($\mathcal{O}^{(t)}$), such as end-to-end latency and resource occupancy.

Storing complete pre- and post-transition snapshots in individual files provides significant operational flexibility for dataset curation. In particular, it enables researchers to randomly sample, partition, or shuffle individual trace files across large simulation runs, whether for stochastic mini-batch training in deep imitation learning or for constructing offline replay buffers, without needing to parse or reconstruct sequential dependencies from a monolithic log stream.

Because these traces retain the full simulation context, the resulting ML and RL tasks can be formulated from multiple perspectives. This flexibility allows the same trace repository to support different learning formulations without regenerating the underlying simulation data. For example, one specific way of exploiting this flexibility is to interpret each JSON record as a Markov Decision Process (MDP) transition tuple [101]:

$$\mathcal{T}_k = (S_k, A_k, R_k, S_{k+1}) \quad (20)$$

Under this perspective, the JSON fields can be associated with the elements of an RL or imitation-learning problem as summarized in Table 5 and detailed below [102]:

- **Decision State (S_k):** The state observed by the learning agent at step k can be defined as the composite tuple $S_k = (\mathcal{G}_k, \mathcal{A}_k, \mathcal{U}_k, \mathcal{P}_k, \mathcal{E}_k)$, where immediately prior to the decision state is captured in terms of the scenarion state (respectively, the physical infrastructure, the applications, the users, and the placement decison) and the exogenous system event ($\mathcal{E}_k$) that triggered the orchestration cycle.
- **Learning Action (A_k):** In this particular mapping, the action can be defined as the orchestration response to the observed state and event. For example, A_k may correspond to the placement or migration plan encoded by ($\Delta\mathcal{P}^{(t)}$). In an imitation-learning setting, the same fields can instead be interpreted as the ground-truth label (Y_k) produced by the reference solver.
- **Future State (S_{k+1}):** The next state corresponds to the continuum configuration after applying the selected orchestration action, and is reconstructed from the post-execution dictionaries.

But as it was mentioned, other mappings are also possible. For instance, a learning task may treat the simulator-generated events ($\mathcal{E}_k$) themselves as the action sequence to be predicted ((A_k)) [103].

The reward R_k of the transition tuple $\mathcal{T}_k$ could be calculated as a mathematical function of the quantitative pre- and post-optimization metrics ($\mathcal{O}^{(t-\Delta t)}$, $\mathcal{O}^{(t)}$). By exposing the granular pre- and post-decision physical metrics rather than a prescriptive scalar reward, the trace architecture remains entirely agnostic to the ML designer's reward shaping. Researchers can freely formulate, weight, or constrain custom reward functions:

$$R_k = f(\Delta o_1, \dots, \Delta o_i), o_i \in \mathcal{O}^{(t-\Delta t)}, \mathcal{O}^{(t)} \quad (21)$$

during training without altering or re-simulating the underlying trace dataset. Note that his flexibility in reward definition is enabled by a distinctive architectural choice in our trace representation by the intentional omission of a hardcoded scalar reward value (R_k) within the exported JSON records. Instead of prescribing a single reward signal, the traces expose the raw, multi-objective telemetry captured after action execution, which is calculated by the solver [104].

This decoupling is essential to preserve the generality and reusability of the dataset across diverse ML research paradigms [105]. In real-world Computing Continuum orchestration, the definition of optimality is highly objective-dependent: one practitioner may seek to strictly minimize network latency, another may prioritize energy conservation and RAM footprint, while another may introduce penalties for service migration overhead.

TABLE 5 | Mapping of Formal MDP Transition Elements to Simulator JSON Trace Fields

RL Concept (MDP Element)	Output Trace	JSON Labels Mapping	Engineering & ML Rationale
Observation / State (S_k)	$\mathcal{U}_k$	users_before dictionary	Combines pre-decision topology graph attributes with the exogenous event triggering orchestration.
	$\mathcal{A}_k$	apps_before dictionary	
	$\mathcal{U}_k$	users_before dictionary	
	$\mathcal{G}_k$	node_before/edge_before dictionaries	
	$\mathcal{P}_k$	placement_before dictionary	
	$\mathcal{E}_k$	action dictionary	
Agent Action (A_k)	$\Delta\mathcal{P}_{k+1}$	diff_message dictionary	Represents optimal service-to-node deployment differences computed by the ILP solver (migration plan). Reflects the updated system topology and physical allocations after placement execution.
	$\mathcal{P}_k$	placement_before dictionary	
	$\mathcal{P}_{k+1}$	placement_after dictionary	
Next State (S_{k+1})	$\mathcal{U}_{k+1}$	users_after dictionary	Raw multi-objective telemetry decoupled from reward shaping to ensure training flexibility.
	$\mathcal{A}_{k+1}$	apps_after dictionary	
	$\mathcal{U}_{k+1}$	users_after dictionary	
	$\mathcal{G}_{k+1}$	node_after/edge_before dictionaries	
	$\mathcal{P}_{k+1}$	placement_after dictionary	
Reward Signal (R_k)	$f(\mathcal{M}_k, \mathcal{M}_{k+1})$	metrics_after/metrics_before dictionaries	

4 | Experimental Evaluation and Results

This section presents a set of representative configurations for generating datasets that emulate realistic Computing Continuum scenarios. We first introduce the general and common setup for all the experiments, that are built from statistical models widely used in the literature and relying only on the simulator's basic event catalog. We then show how ECCOS can be exploited more effectively by simulating realistic situations that require more complex state modifications, implemented through composed events in the simulator. For each dataset or experiment, we also analyze the temporal evolution of selected metrics in order to illustrate how this composed events influences in the simulated scenario and how the changes over time capture such dynamics.

It is important to clarify that the contributions of this work center on the development of a simulator for dataset generation rather than on the design of new optimization algorithms. These contributions include a modular architecture decoupled from the placement solver, fine-grained experiment specification through YAML configuration files, and the composition of user-defined complex events from existing event primitives through the same configuration mechanism, without requiring changes to the source code. The simulator also supports the dynamic evolution of the simulated environment throughout execution, allowing infrastructure conditions, application characteristics, and user behavior to change as events occur.

Accordingly, the experiments are designed to demonstrate these capabilities and illustrate how they support the generation of datasets that capture distinct forms of system dynamics. Developing solvers that advance the state of the art or conducting a comparative evaluation of service placement optimization algorithms falls outside the scope of this work. Instead, the results focus on the events generated during each simulation and their effects on the specific metrics affected by those events. The metrics examined therefore differ across the three selected experiments, reflecting the distinct phenomena represented in each scenario.

The three experiments presented here are illustrative examples of how ECCOS can be configured to model real-world events affecting Computing Continuum infrastructures, rather than an exhaustive account of its modeling capabilities. The simulator is not restricted to these scenarios: its extensive catalog of base events and its support for user-defined composed events enable a wide variety of situations to be represented within these infrastructures. The resulting range of possible event combinations and configurations is too broad to cover exhaustively in a single study; the selected experiments therefore demonstrate representative uses of these capabilities rather than define their limits.

The configuration files for the baseline and the three scenarios examined in this section are available in the code and data repository. The repository also includes the JSON output files generated by executing these scenarios, with each simulation comprising a total of 5000 events, to facilitate reproducibility and further analysis of the generated traces.

TABLE 6 | Configurable aspects via statistical distributions in ECCOS, optimal modeling, and bibliographic justification.

Configurable Aspect and YAML Path	Optimal Distribution	Realistic Parameters	Bibliographic Reference
Node Capacity (RAM) + <code>infrastructure.node.attributes.ram.distribution</code>	Lognormal	$\mu = 0.1, \sigma = 0.5$	[106]
Edge Capacity (Delay) + <code>infrastructure.edge.attributes.delay.distribution</code>	Lognormal	$\mu = 10.0, \sigma = 2.0$	[107]
Failure Frequency (Nodes / Edges) + <code>infrastructure.node.actions.disable_node.frequency</code>	Weibull	shape = 0.8 scale = 300.0	[108]
Recovery Time (MTTR) + <code>node.actions.disable_node.impact.distribution_to_enable_node</code>	Lognormal	$\mu = 2.5, \sigma = 1.2$	[108]
Node/Network Degradation Magnitude + <code>node.actions.degrade_node.impact.p_loss_dist</code>	Beta	$\alpha = 2.0, \beta = 5.0$	[109]
Microservice Resource Demand (RAM) + <code>app.services.profiles.<profile>.attributes.ram.distribution</code>	Lognormal	$\mu = 2.0, \sigma = 1.0$	[110]
Global Application Popularity + <code>app.popularity.distribution</code>	Zipf	$a = 1.2$	[111]
Popularity Surge Duration + <code>app.actions.surge_popularity.impact.duration_dist</code>	Weibull	$a = 1.5$ scale = 10.0	[112]
Resource Footprint Variation (Noise) + <code>app.actions.update_app_footprint.impact.sigma_footprint</code>	Uniform	low = 0.05 high = 0.15	[113]
Geographic Demand Drift + <code>app.actions.geo_demand_shift.impact.drift_dist</code>	Normal	$\mu = 0.0, \sigma = 0.05$	[114]
User Movement Speed + <code>user.mobility.speed</code>	Uniform	low = 1.0 high = 1.5 (m/s)	[114]
Wireless Coverage Radius + <code>user.mobility.coverage_radius</code>	Lognormal	$\mu = 0.2, \sigma = 0.02$	[115]
Initial Request Rate + <code>user.request_ratio</code>	Exponential	scale = 5.0	[116]
Temporary Disconnection Duration + <code>user.actions.suspend_user.impact.distribution_to_resume_user</code>	Lognormal	$\mu = 1.0, \sigma = 0.5$	[117]
New Users Arrival + <code>global_spawner.actions.new_user.frequency</code>	Exponential	scale = 30.0	[118]
New Apps Arrival + <code>global_spawner.actions.new_app.frequency</code>	Exponential	scale = 1000.0	[118]

TABLE 7 | Scenario configuration aspects defined in ECCOS by selecting implemented alternatives or by specifying specific parameter values, including bibliographic justification.

Configurable Aspect and YAML Path	Optimal Alternative	Realistic Parameters	Bibliographic Reference
Infrastructure topology + <code>infrastructure.model.name</code>	multi_tier	cloud.percentatge:2 fog.percentatge:10	[119, 120, 121, 122, 123]
Node and edge attributes model + <code>attributes.<attribute>.mode</code>	centrality_based	centrality_type: direct_proportional	[124]

Each experiment was defined by starting from a common baseline configuration, adding composed events, and modifying selected baseline parameters to represent the intended scenario. Table 6 summarizes the statistical distributions used to configure the simulator’s base events and other stochastic aspects of this baseline. The configurations presented in the tables are based on approaches widely used in scientific studies, and the corresponding bibliographic references are included to justify their selection. The scenario-specific additions and parameter changes are described in the following subsections.

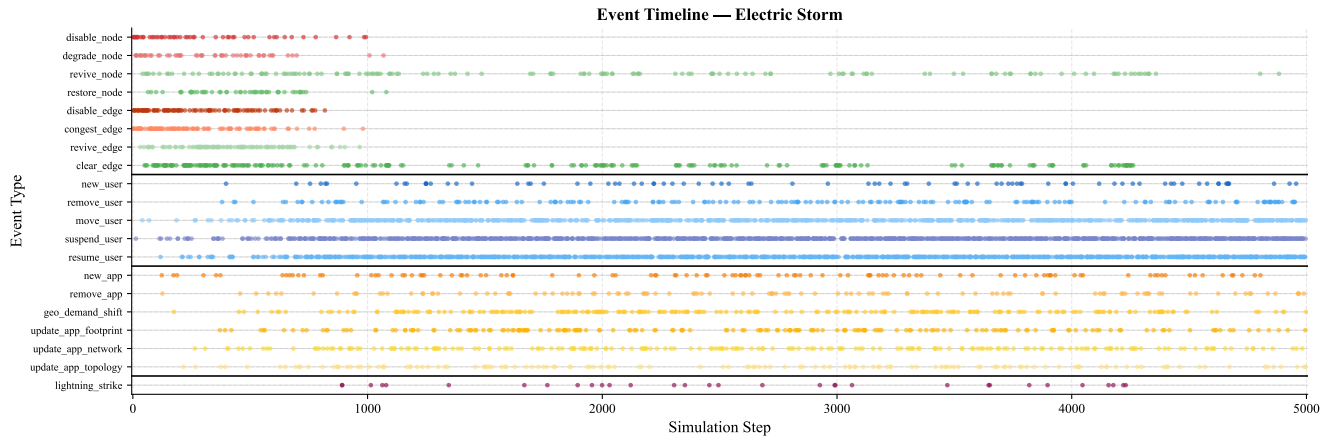


FIGURE 6 | Event timeline for the electrical-storm simulation. Event types are grouped along the y-axis, and each marker indicates that an event was triggered at a specific simulation step.

4.1 | Transient Infrastructure Disruption Scenario

To evaluate the capacity of ECCOS to represent realistic, temporary, and spatially correlated infrastructure disruptions, we model a severe electrical storm affecting the network region. This scenario is intended to reproduce the combined consequences of lightning strikes, power surges, and local electrical instability, which may simultaneously interrupt computing nodes, disable communication links, and temporarily reduce the capacity of infrastructure components that remain operational. Unlike the randomized baseline, where events occur independently according to their individual stochastic processes, the storm scenario introduces causal and spatial relationships among multiple infrastructure effects.

The background failure processes are configured to exhibit bursty behavior. Node failures generated through `disable_node` follow a Weibull inter-arrival distribution with shape 0.8 and scale 60, whereas edge outages generated through `disable_edge` use the same shape parameter and a shorter scale of 45. A Weibull shape below one produces a decreasing hazard rate and clusters failures into periods of increased activity, which is consistent with empirical observations showing that infrastructure failures are neither uniformly distributed nor independent over time [108, 125].

The storm also produces partial damage that does not necessarily result in complete disconnection. Power fluctuations and transient surges are represented through `degrade_node` events whose degradation intensity follows a Beta distribution with parameters $\alpha = 2$ and $\beta = 2$. When mapped onto the configured high-severity degradation range, this symmetric distribution generates a broad set of moderate-to-severe capacity losses while avoiding an artificial concentration at either extreme. Consequently, the scenario combines blackouts, link outages, and brownout-like reductions in computational capacity, thereby creating a more demanding and heterogeneous infrastructure state than a model based exclusively on binary failures.

Because each infrastructure component initializes an independent degrade and failure clock at the beginning of the simulation, this configuration produces a comparatively high concentration of failures and degradation during the initial simulation period, concentrating them around the 1000 first simulation steps. In addition, this temporal pattern enables the effects of infrastructure failures to be compared with the comparatively failure-free period of the simulation spanning events 1000 to 5000. This behavior can be observed in Figure , which shows all the events generated throughout the electrical-storm simulation.

The principal storm effect is implemented as the composed event `lightning_strike`, defined as a macro-event in the `global_spawner`. Its occurrence follows an exponential distribution with scale 90, representing the arrival of individual lightning strikes during the storm. Each occurrence triggers several primitive actions within the same simulation step: it disables a group of three nearby nodes selected through the `self_proximity` strategy and congests five nearby edges. This composition models a single physical cause that produces multiple, spatially related consequences, rather than approximating the strike through unrelated independent events. It therefore demonstrates how the composed-event mechanism can express correlated disruptions while reusing the basic node-failure and edge-congestion actions already provided by the simulator. This can be also observed in Figure 4.1, where `lightning_strike` is triggered along the simulation.

The overall effect of the storm on infrastructure availability is illustrated in Figure 7, which reports the number of operational nodes throughout the simulation. The electric storm plot exhibits the initial stage with a widespread reduction in node availability associated with the concentration of `disable_node` events. This general disruption is accompanied by intermittent losses affecting smaller subsets of nodes at later simulation steps, corresponding to the repeated execution of

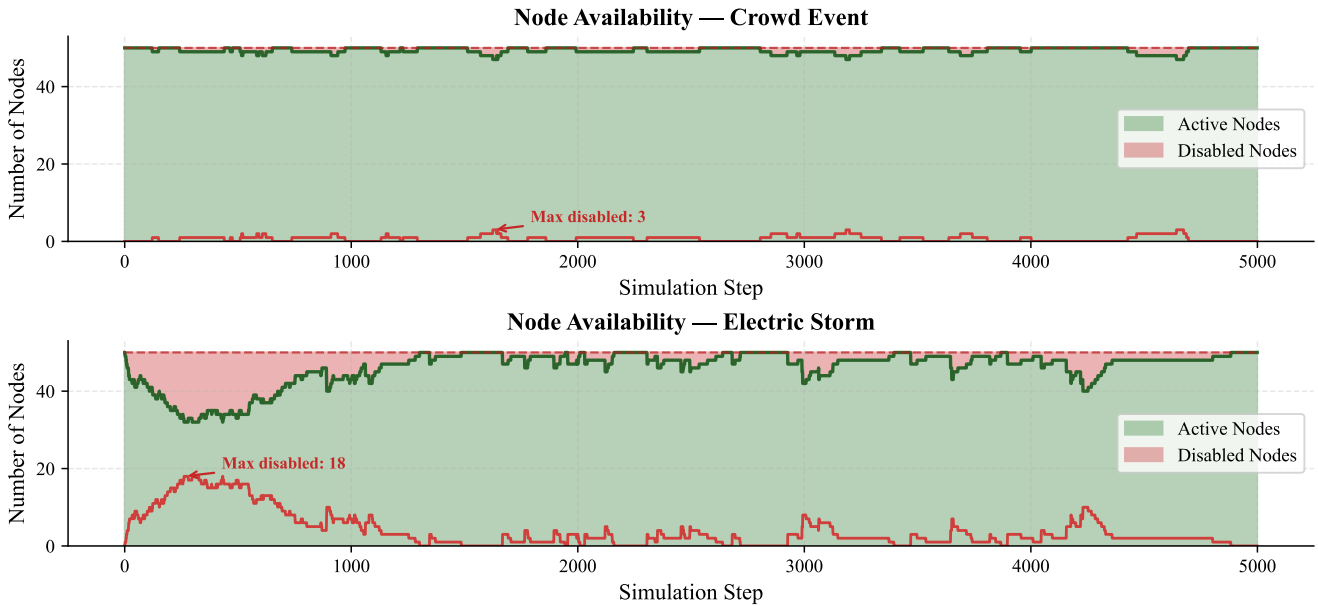


FIGURE 7 | Node availability along the simulation steps for electric storm and crowding event experiments.

the composed `lightning_strike` event. The distinctive impact of the storm becomes clearer when this availability profile is compared with the crowded event plot for the application demand surge scenario. In that experiment, infrastructure failures remain much more infrequent and are distributed more homogeneously throughout the simulation, providing a reference case in which changes in system behavior are primarily driven by workload growth rather than by concentrated infrastructure disruption.

4.1.1 | Application Demand Surge Scenario

To evaluate the capacity of ECCOS to represent application-driven stress and exponential demand growth, we model an application demand surge scenario. This scenario is intended to reproduce the consequences of a popular application going viral (e.g., breaking news, or a trending social media challenge), which causes an explosive increase in demand that propagates across the network. Unlike infrastructure-centric disruptions, this scenario introduces extreme application dynamics where popularity surges lead to rapid footprint growth, intensified user activity, and subsequent network congestion.

The application and user behaviors are configured to exhibit highly dynamic and extreme patterns. Popularity surges generated through `surge_popularity` follow an exponential inter-arrival distribution with scale 30, producing frequent spikes in demand as viral content propagates. To reflect auto-scaling mechanisms under viral load, app footprint changes through `update_app_footprint` and network variations through `update_app_network` are also triggered frequently, with exponential scales of 50 and 60, respectively. Concurrently, user behavior is heavily modified by `change_request_ratio` events, which have an 80% probability of increasing the request rate with a multiplier up to 5.0, modeling the binge-consumption of viral content.

Because application and user events are highly active throughout the execution, this configuration produces a dense and continuous concentration of demand-driven events across the entire simulation. This temporal pattern enables the effects of application stress to be compared with scenarios where the workload remains relatively stable. This behavior can be observed in Figure 8, which shows all the events generated throughout the demand surge simulation.

The principal viral effect is implemented as the composed event `viral_cascade`, defined as a macro-event in the `global_spawner`. Its occurrence follows an exponential distribution with scale 60, representing periodic re-sharing waves of viral content through social network layers. Each occurrence triggers several primitive actions within the same simulation step: it multiplies the popularity of two applications (which automatically increases the request ratio of all users subscribed to them), forces a footprint update for two applications, and congests three random edges. This composition accurately models the cascading nature of viral demand, where popularity spikes lead directly to resource scaling and network congestion, rather than approximating the surge through unrelated independent events. It therefore demonstrates how the composed-event mechanism can express complex, multi-domain phenomena.

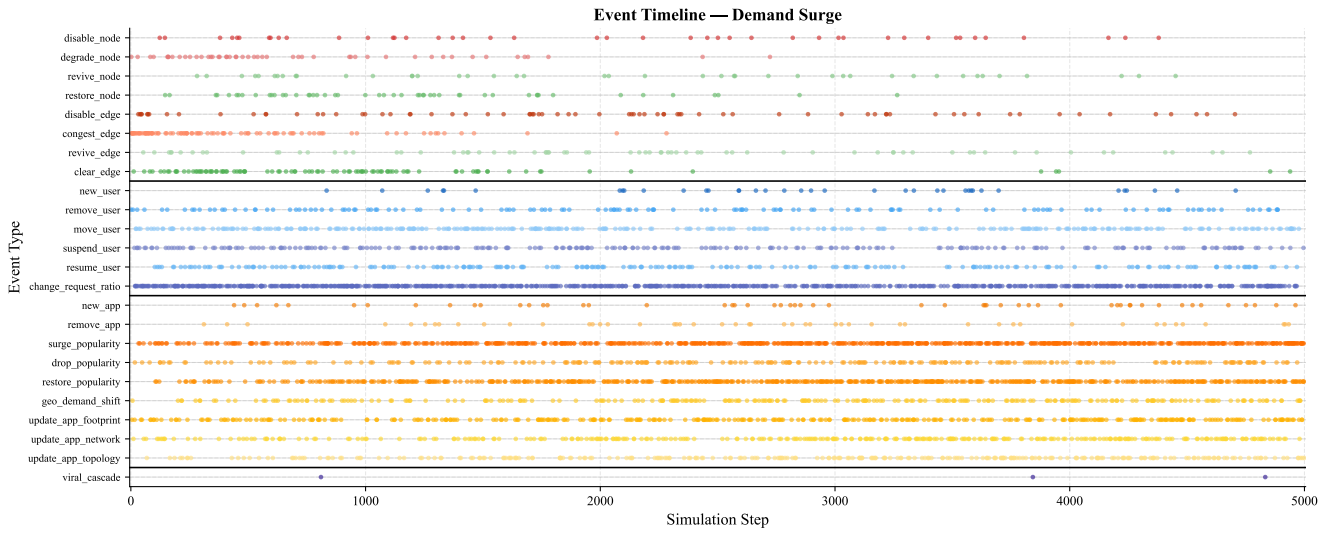


FIGURE 8 | Event timeline for the application demand surge simulation. Event types are grouped along the y-axis, and each marker indicates that an event was triggered at a specific simulation step.

The overall effect of the demand surge on the system state is illustrated in Figure 9, which shows the evolution of aggregated application request rates throughout the simulation. For the specific of the demand surge experiment, the figure includes an additional lower subplot that provides a zoom view of the less frequently requested applications by excluding the viral ones. This additional view is necessary because the viralization events in the demand surge experiment produce substantial increases in the request ratios of the affected applications, making the much smaller request rates of the remaining applications difficult to distinguish on the same scale.

The distinctive impact of the demand surge becomes clearer when this request-rate profile is compared with that of the electrical-storm scenario, which intentionally excludes events that directly modify application request rates. In the electrical-storm scenario, the most frequently requested applications have request rates comparable to those of the less frequently requested applications. Although request rates also vary in this experiment, these changes arise from variations in entity availability and user dynamics rather than from explicit application demand events. For example, requests from unavailable users no longer contribute to the aggregate metric, while user arrivals, departures, and movements outside the coverage area also affect the total request rate. Overall, these variations are more homogeneous in magnitude and more uniformly distributed over the course of the simulation.

In contrast, the demand surge scenario exhibits exceptionally high request rates for the viral applications during the periods associated with application surge events. These rates increase in successive steps until the corresponding viral episode ends, producing the staircase-like pattern observed in the demand surge plot.

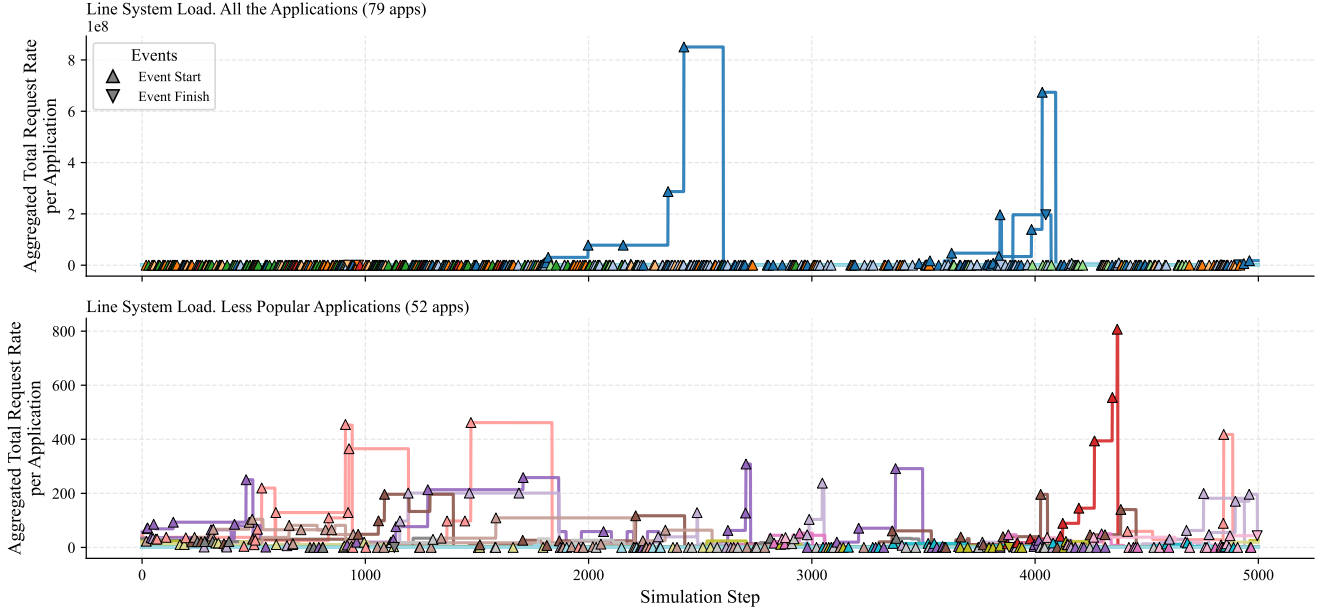
Because application popularity follows a Zipf distribution, a small subset of applications accounts for most of the system traffic. Large increases in the aggregate request rate, such as those observed around events 2000 and 4000, occur when demand-increasing events affect these highly popular applications. Such increases can result from either composed events or independent base events, including popularity surges and user request increments, and therefore do not necessarily coincide with the composed-event timeline. Since randomly selected targets are more likely to belong to the many less popular applications, most events have a much smaller effect on aggregate demand.

4.2 | Localized mass gathering of people at events scenario

To evaluate the capacity of ECCOS to represent extreme spatial concentration of demand and synchronized behavioral responses, we model a localized people gathering in a hot spot. This scenario is intended to reproduce the consequences of large-scale public gatherings, such as football matches or music concerts, which cause tens of thousands of individuals to converge on a single geographic location and simultaneously consume data-intensive content. Unlike infrastructure-centric disruptions or general demand surges, this scenario introduces extreme spatial concentration and bursty, correlated temporal dynamics where demand is funneled through a small number of local edge nodes.

The infrastructure and user behaviors are configured to exhibit highly concentrated and demanding patterns. Node degradation generated through `degrade_node` follows an exponential inter-arrival distribution with scale 30, severely reducing capacity via a Beta distribution ($\alpha = 3, \beta = 4$) to model thermal throttling and memory pressure. To reflect

Per-Application Aggregated Request Rate — Demand Surge



Per-Application Aggregated Request Rate — Electric Storm

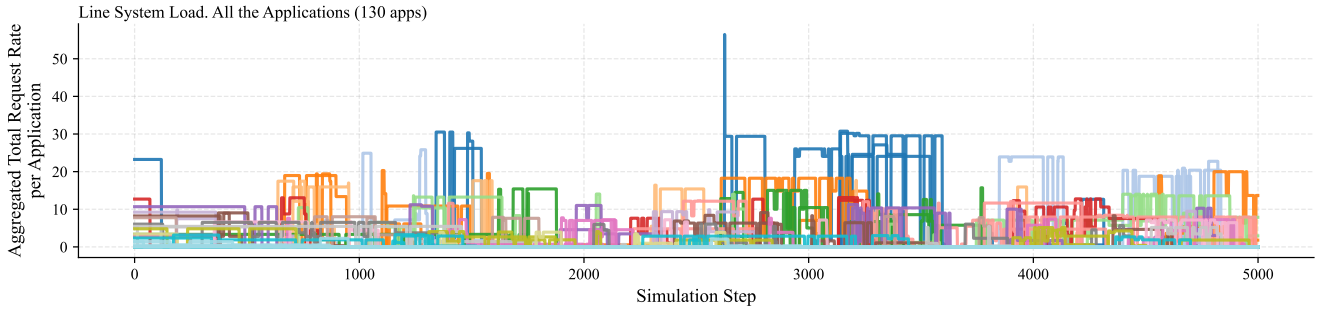


FIGURE 9 | Aggregated request rate for each application in the system over the simulation steps for the electrical-storm and application demand surge experiments. Icons on the curves mark the start and end of each application demand surge event.

the saturation of local cell towers, edge congestion events triggered through `congest_edge` occur frequently with an exponential scale of 20, producing significant bandwidth losses and delay inflation. Concurrently, user behavior is heavily dynamic; `change_request_ratio` events frequently increase service request rates with multipliers up to 5.0, and `surge_popularity` events occur with an exponential scale of 35 to model localized viral moments. Because users are tightly clustered around a single geographic hotspot ($\sigma_s = 0.015$) and infrastructure degradation is highly localized, this configuration produces a dense and continuous concentration of congestion and degradation events funneled through a small subset of edge nodes.

The principal synchronized reaction is implemented as the composed event `crowd_surge`, defined as a macro-event in the `global_spawner` (Figure 10). Its occurrence follows an exponential distribution with scale 80, representing significant moments during the event that trigger coordinated audience reactions, such as a goal in a match. Each occurrence triggers several primitive actions within the same simulation step: it injects a burst of 20 new users via `new_user`, spikes the popularity of 3 local applications via `surge_popularity`, congests 4 nearby network links, and immediately spikes the request ratios of 8 users. This composition accurately models the multi-dimensional shock of crowd-driven disruptions, where new arrivals, popularity shifts, and synchronized demand spikes occur simultaneously, rather than approximating the surge through unrelated independent events. It therefore demonstrates how the composed-event mechanism can express correlated, multi-modal perturbations.

The overall effect of the crowd surge on the infrastructure is characterized by localized high-demand connections concentrated on a small subset of edge nodes and by extreme spikes in user request rates (Figure 11). The distinctive impact of this scenario becomes clearer when the evolution of the geographical concentration of users is analyzed alongside the

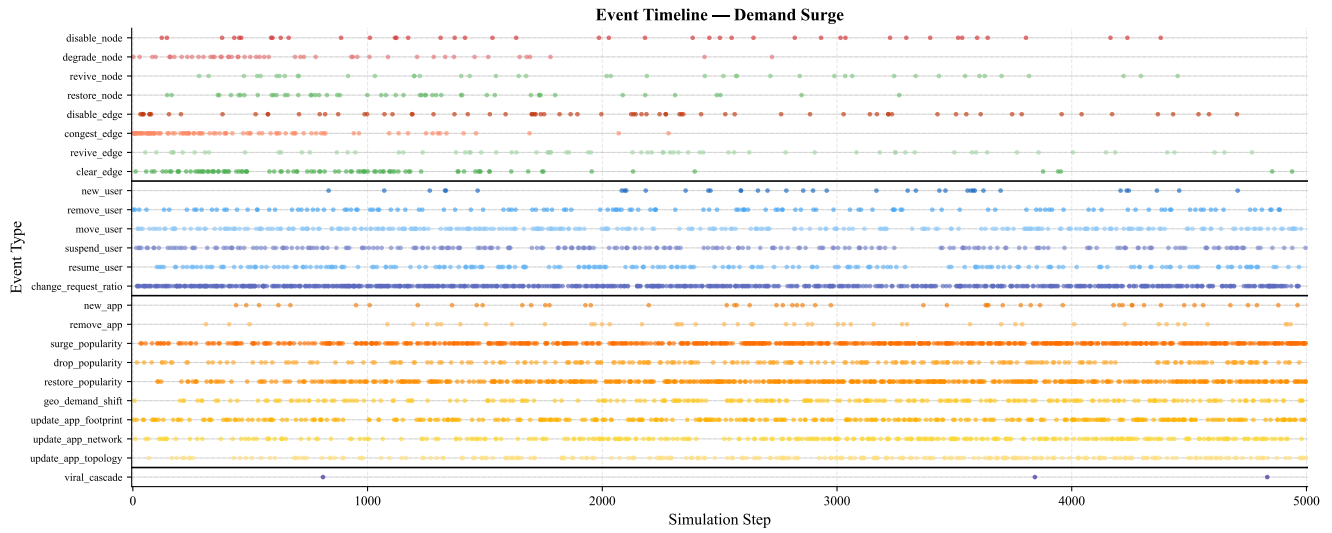


FIGURE 10 | Event timeline for the localized crowd event simulation. Event types are grouped along the y-axis, and each marker indicates that an event was triggered at a specific simulation step.

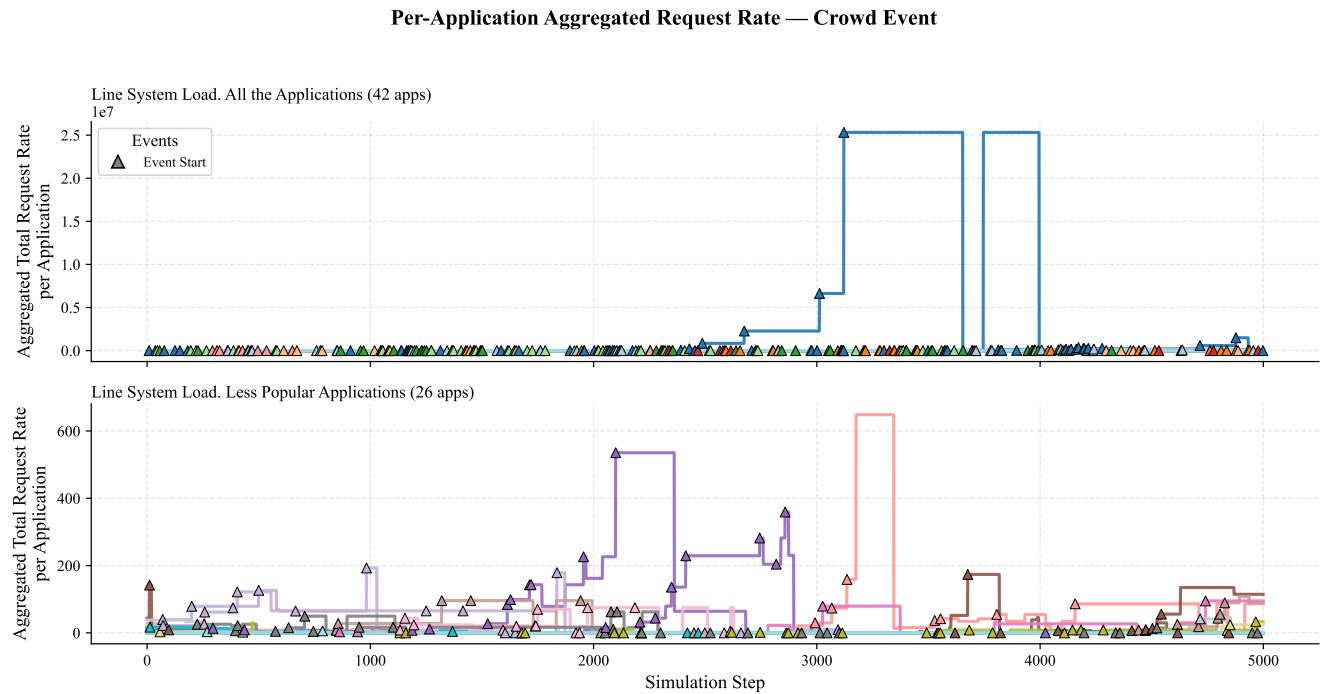


FIGURE 11 | Aggregated request rate for each application in the system over the simulation steps for the crowd-event experiment. Icons on the curves mark the start and end of each application demand surge event.

number of users connected to each node following crowd_surge events. Figure 12 shows the geographical distribution of users within the simulation area and statistics on the number of users connected to edge nodes at four selected simulation steps, immediately after several crowd_surge events. The hotspot becomes increasingly concentrated after each composed event, as indicated by the progressively more intense red regions in the heatmaps, while nearby edge nodes serve a growing number of connected users. These results demonstrate the simulator's ability to represent localized mass-gathering events, generate the associated surges in application requests, and support the evaluation of their potential effects on system performance.

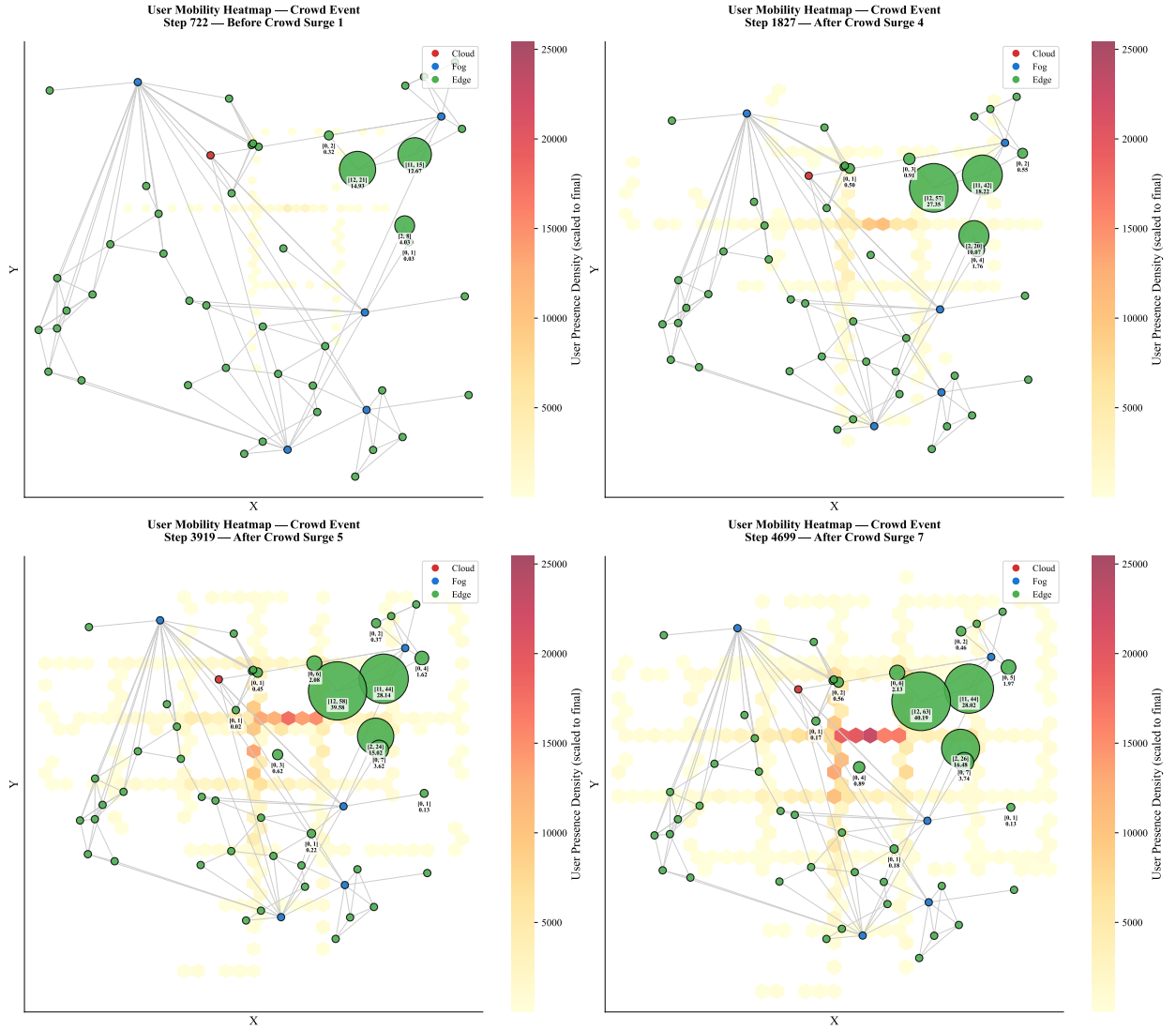


FIGURE 12 | Evolution of user distribution during the crowd-event experiment. The four panels show heatmaps of user positions at selected simulation steps together with the number of users connected to each edge node, summarized by the minimum, maximum, and mean values.

5 | Conclusions

In this paper, we presented ECCOS (Event-Driven Computing Continuum Orchestration Solver), a highly modular, macroscopic discrete-event simulation framework designed to advance research in Computing Continuum resource orchestration and data-driven management. ECCOS directly overcomes the structural rigidities, computational bottlenecks, and poor data interoperability of traditional network simulators by decoupling stochastic event scheduling, state management, algorithmic placement solvers, and telemetry export into distinct operational components.

Through its declarative YAML specification schema and domain-isolated pseudorandom streams, ECCOS ensures strict experimental reproducibility while enabling the fine-grained modeling of multi-tier topologies, realistic user mobility patterns, and complex microservice dependency graphs (both SFCs and DAGs). A key innovation of the framework lies in its composable macro-event engine, which seamlessly translates, for example, high-level physical disruptions into spatially correlated primitive state transitions across infrastructure, user, and application domains.

Our experimental evaluation demonstrated the capacity of ECCOS to capture complex spatio-temporal dynamics across multi-tier continuum deployments. By evaluating scenarios involving concentrated electrical storms and explosive viral demand surges, we showed how ECCOS dynamically tracks system behavior, evaluates placement solver performance, and seamlessly absorbs localized stress without introducing simulation bottlenecks.

Crucially, ECCOS bridges the gap between network simulation and Artificial Intelligence. Via the ECCO-BRIDGE module, every orchestration cycle is automatically formatted into self-contained, graph-structured JSON records that map directly onto Markov Decision Process (MDP) transition tuples. By decoupling reward calculation from raw telemetry export,

ECCOS provides a versatile data ecosystem where researchers can flexibly define multi-objective reward functions without re-executing long simulation runs.

Several promising avenues remain for extending the ECCOS framework. The ECCO-SOLVER architecture could be extended to support multi-agent reinforcement learning (MARL), allowing distributed edge domain managers to negotiate service migration in a federated, privacy-preserving manner. Another direction involves enhancing the ECCO-BRIDGE telemetry protocol to ingest real-time state updates directly from real-world Kubernetes/K3s edge clusters, enabling live synchronization and hybrid digital-twin closed-loop testing. Further extensions could enable automatic third-party integrations, such as Farama Gymnasium, allowing ECCOS scenarios to be exposed as reinforcement learning environments for training and evaluating orchestration agents.

In summary, ECCOS offers a scalable, flexible, and mathematically grounded open-source platform that enables the scientific community to benchmark novel orchestration policies and generate the massive synthetic datasets needed to power the next generation of AI-driven Cloud-Fog-Edge systems.

Author Contributions

Mireia Jaume: Data Curation, Formal analysis, Methodology, Software, Validation, Visualization, Writing - review & editing. **Isaac Lera:** Conceptualization, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Supervision, Writing - review & editing. **Carlos Guerrero:** Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing - original draft.

Acknowledgments

The authors acknowledge the use of GPT-5.2 (accessed june-july 2026) to support improve language correction and to correct language typos. All AI-generated suggestions were reviewed, revised, and approved by the authors, who take full responsibility for the accuracy and integrity of the work.

The authors acknowledge the use of Gemini 3.5 Flash and Gemini 3.1 Pro (accessed may-june 2026) for partial refactoring tasks in the source code of ECCOS and for the detection of errors in the software testing phase.

Financial Disclosure

This work was supported by Grant PID2024-158637OB-I00, funded by MICIU/AEI/10.13039/501100011033 and by “ERDF A way of making Europe”, “ERDF/EU”.

Supporting Information

The data generated in this study and the source code of ECCOS are available in a public repository at <https://github.com/acsicuib/XXXXXXX/>, under the MIT License.

Conflicts of Interest

The authors declare no conflicts of interest.

References

1. Auday Al-Dulaimy, Matthijs Jansen, Bjarne Johansson, Animesh Trivedi, Alexandru Iosup, M. Ashjaei, A. Galletta, Dragi Kimovski, R.-C. Prodan, Konstantinos Tserpes, George Kousiouris, Chris Giannakos, Ivona Brandic, Nawfal Ali, André B. Bondi,, and Alessandro V. Papadopoulos. “The computing continuum: From IoT to the cloud.” *Internet Things*27 (2024): 101272.
2. Waseem Sajjad, Montse Farreras, Jordi Garcia,, and X. Masip-Bruin. “Service placement in the continuum: A systematic literature review.” *Comput. Commun.*246 (2026): 108370.
3. G. Walia, Mohit Kumar,, and S. Gill. “AI-Empowered Fog/Edge Resource Management for IoT Applications: A Comprehensive Review, Research Challenges, and Future Perspectives.” *IEEE Communications Surveys Tutorials*26 (2024): 619–669.
4. Mihaela Mihaiu, Bogdan-Costel Mocanu, Cătălin Negru, Alina Petrescu-Nita,, and Florin Pop. “Resource Allocation Optimization Model for Computing Continuum.” . *Mathematics*.
5. Fatima Bendaouch, Hayat Zaydi, Safae Merzouk,, and Saliha Assoul. “Benchmarking IoT Simulation Frameworks for Edge–Fog–Cloud Architectures: A Comparative and Experimental Study.” *Future Internet* 17, no. 9.
6. Jianpeng Qi, Chao Liu, Xiao Zhang, Lei Wang, Rui Wang, Junyu Dong,, and Yanwei Yu. “A Survey on Open-Source Edge Computing Simulators and Emulators: The Computing and Networking Convergence Perspective.” *ArXiv*abs/2505.09995.
7. Sandipkumar Patel, and Ritesh Patel. “Fog Computing: A Comprehensive Analysis of Simulation Tools, Applications and Research Challenges with Use Cases..” *Journal of Engineering Science & Technology Review* 15, no. 3.

8. Rajkumar Buyya, Rajiv Ranjan, and Rodrigo N. Calheiros. 2009. "Modeling and simulation of scalable Cloud computing environments and the CloudSim toolkit: Challenges and opportunities." In *2009 International Conference on High Performance Computing Simulation*, 1–11.
9. Harshit Gupta, Amir Vahid Dastjerdi, Soumya K. Ghosh, and Rajkumar Buyya. "iFogSim: A toolkit for modeling and simulation of resource management techniques in the Internet of Things, Edge and Fog computing environments." *Software: Practice and Experience* 47, no. 9 (2017): 1275–1296.
10. Cagatay Sonmez, Atay Ozgovde, and Cem Ersoy. "EdgeCloudSim: An environment for performance evaluation of edge computing systems." *Transactions on Emerging Telecommunications Technologies* 29, no. 11 (2018): e3493. e3493 ett.3493.
11. Charafeddine Mechali, Hajer Taktak, and Faouzi Moussa. "PureEdgeSim: A simulation framework for performance evaluation of cloud, edge and mist computing environments." *Comput. Sci. Inf. Syst.* 18 (2020): 43–66.
12. Isaac Lera, Carlos Guerrero, and Carlos Juiz. "YAFS: A Simulator for IoT Scenarios in Fog Computing." *IEEE Access* 7 (2019): 91745–91758.
13. Tariq Qayyum, Asad Waqar Malik, Muazzam A. Khan Khattak, Osman Khalid, and Samee U. Khan. "FogNetSim++: A Toolkit for Modeling and Simulation of Distributed Fog Environment." *IEEE Access* 6 (2018): 63570–63583.
14. A. Brogi, and S. Forti. "QoS-aware Deployment of IoT Applications Through the Fog." *IEEE Internet of Things Journal* 4, no. 5 (2017): 1185–1192.
15. Monika Gill, and Dinesh Singh. "A comprehensive study of simulation frameworks and research directions in fog computing." *Computer Science Review* 40 (2021): 100391.
16. Andras Markus, and Attila Kertesz. "A survey and taxonomy of simulation environments modelling fog computing." *Simulation Modelling Practice and Theory* 101 (2020): 102042. Modeling and Simulation of Fog Computing.
17. Spiridoula V. Margariti, Vassilios V. Dimakopoulos, and Georgios Tsoumanis. "Modeling and Simulation Tools for Fog Computing—A Comprehensive Survey from a Cost Perspective." *Future Internet* 12, no. 5.
18. Sergej Svorobej, Patricia Takako Endo, Malika Bendeche, Christos Filelis-Papadopoulos, Konstantinos M. Gian-noutakis, George A. Gravvanis, Dimitrios Tzovaras, James Byrne, and Theo Lynn. "Simulating Fog and Edge Computing Scenarios: An Overview and Research Challenges." *Future Internet* 11, no. 3.
19. Carlo Puliafito, Diogo M. Gonçalves, Márcio M. Lopes, Leonardo L. Martins, Edmundo Madeira, Enzo Mingozzi, Omer Rana, and Luiz F. Bittencourt. "MobFogSim: Simulation of mobility and migration for fog computing." *Simulation Modelling Practice and Theory* 101 (2020): 102062. Modeling and Simulation of Fog Computing.
20. Prince Gupta, Rajeev Sharma, and Sachi Gupta. "Simulators for fog computing and information processing." *Proceedings of the National Academy of Sciences, India Section A: Physical Sciences* 94, no. 4 (2024): 437–447.
21. Özgür Umut Akgül, Wencan Mao, Byungjin Cho, and Yu Xiao. "VFogSim: A Data-Driven Platform for Simulating Vehicular Fog Computing Environment." *IEEE Systems Journal* 17 (2023): 5002–5013.
22. Dongheon Lee, Sheng Zhou, Xiaofeng Zhong, Z. Niu, Xuan Zhou, and Honggang Zhang. "Spatial modeling of the traffic density in cellular networks." *IEEE Wireless Communications* 21 (2014): 80–88.
23. André Felipe Zanella, Antonio Bazco-Nogueras, Cezary Ziemlicki, and Marco Fiore. "Characterizing and Modeling Session-Level Mobile Traffic Demands from Large-Scale Measurements." . *Proceedings of the 2023 ACM on Internet Measurement Conference*.
- 24.
25. S. Jun, P. Canal, J. Apostolakis, A. Gheata, and L. Moneta. "Vectorization of random number generation and reproducibility of concurrent particle transport simulation." *Journal of Physics: Conference Series* 1525.
26. Roberto Casadei, G. Fortino, Danilo Pianini, Andrea Placuzzi, Claudio Savaglio, and M. Viroli. "A Methodology and Simulation-Based Toolchain for Estimating Deployment Performance of Smart Collective Services at the Edge." *IEEE Internet of Things Journal* 9 (2022): 20136–20148.
27. J. Norman. "Connectivity and Coverage in Hybrid Wireless Sensor Networks using Dynamic Random Geometric Graph Model." *International Journal on Applications of Graph Theory in Wireless Ad Hoc Networks and Sensor Networks* 3 (2011): 39–47.
28. R. S. Couto, Stefano Secci, Miguel Elias M. Campista, and L. Costa. "Reliability and Survivability Analysis of Data Center Network Topologies." *Journal of Network and Systems Management* 24 (2015): 346 – 392.
29. Fanchen Bu, Ruochen Yang, Paul Bogdan, and Kijung Shin. "Edge Probability Graph Models Beyond Edge Independency: Concepts, Analyses, and Algorithms." . *2025 IEEE International Conference on Data Mining (ICDM)* (2024): 100–109.
30. Harris K. Armeniakos, P. Bithas, Sotiris A. Tegos, A. Kanatas, and G. Karagiannidis. "Stochastic Geometry for Modeling and Analysis of Sensing and Communications: A Survey." *IEEE Communications Surveys Tutorials* 28 (2025): 2691–2724.
31. R. Rak, and E. Rak. "The Fractional Preferential Attachment Scale-Free Network Model." *Entropy* 22.
32. M. Tosun, U. Çabuk, Elif Haytaoglu, Orhan Dağdeviren, and Y. Ozturk. "CoRMAC: A Connected Random Topology Formation With Maximal Area Coverage in Wireless Ad-Hoc Networks." *IEEE Internet of Things Journal* 10 (2023): 12379–12392.
33. Zhiyang Guo, and Yuanyuan Yang. "On Nonblocking Multicast Fat-Tree Data Center Networks with Server Redundancy." *IEEE Transactions on Computers* 64 (2015): 1058–1073.
34. Niroshinie Fernando, Samir Shrestha, S. Loke, and Kevin Lee. "On Edge-Fog-Cloud Collaboration and Reaping Its Benefits: A Heterogeneous Multi-Tier Edge Computing Architecture." *Future Internet* 17 (2025): 22.
35. S. Sefati, A. Nor, Bahman Arasteh, Răzvan Crăciunescu, and C. Comsa. "A Probabilistic Approach to Load Balancing in Multi-Cloud Environments via Machine Learning and Optimization Algorithms." *Journal of Grid Computing* 23.

36. Yuexin Wang, Jiali You., and Yang Li. "Dynamic Resource Aggregation Method Based on Statistical Capacity Distribution." *Electronics*.
37. S. Oldham, Ben D. Fulcher, Linden Parkes, Aurina Arnatkevic Iütè, Chao Suo., and A. Fornito. "Consistency and differences between centrality measures across distinct classes of networks." *PLoS ONE*14.
38. Y. Kalyani, and Rem W. Collier. "A Systematic Survey on the Role of Cloud, Fog, and Edge Computing Combination in Smart Agriculture." *Sensors (Basel, Switzerland)*21.
39. C. Bettstetter, H. Hartenstein., and X. Costa. "Stochastic Properties of the Random Waypoint Mobility Model." *Wireless Networks*10 (2004): 555–567.
40. Jiaqi Han, F. Zheng, Jingjing Luo., and Xu Zhu. "Local Delay Analysis and Improvement for User-SBS Correlated Deployments With Random DTX." *IEEE Transactions on Vehicular Technology*71 (2022): 13007–13016.
41. Lihua Yang, Junhui Zhao., and Ziyang Zhang. "Comparative Analysis of Distance Distributions for CoMP-Aided Cellular Networks Using Thomas Cluster Process." *IEEE Communications Letters*25 (2021): 3503–3507.
42. Chiranjib Saha, Mehrnaz Afshang., and Harpreet S. Dhillon. "Enriched K -Tier HetNet Model to Enable the Analysis of User-Centric Small Cell Deployments." *IEEE Transactions on Wireless Communications*16 (2016): 1593–1608.
43. S. Kour, and Jagpal Singh. "Performance Evaluation of Enhanced Manhattan Mobility Model Over GM, RWP, Manhattan Grid, SLAW, and TLW Mobility Models in MANETs." *Recent Advances in Computer Science and Communications* 14, no. 7 (2021): 992–1000.
44. C. Bettstetter. "Mobility modeling in wireless networks: categorization, smooth movement, and border effects." *ACM SIGMOBILE Mob. Comput. Commun. Rev.*5 (2001): 55–66.
45. F. Bai, Narayanan Sadagopan., and A. Helmy. 2003. "IMPORTANT: a framework to systematically analyze the Impact of Mobility on Performance of Routing Protocols for Adhoc Networks." In *IEEE INFOCOM 2003. Twenty-second Annual Joint Conference of the IEEE Computer and Communications Societies (IEEE Cat. No.03CH37428)*, Vol 2, 825–835 vol.2.
46. Siham Sadiki, Nisrine Ibadah, Hanae Belmajdoub., and K. Minaoui. "Advanced Clustering Metric for Mobility Model Using DBSCAN." *IEEE Transactions on Intelligent Vehicles*10 (2025): 5249–5261.
47. J. Kerdsri, and T. Veeraklaew. "Visualization of Spatial Distribution of Random Waypoint Mobility Models." *J. Comput.*12 (2017): 309–316.
48. Tomás Cerný, Amr S. Abdelfattah, Jorge Yero., and Davide Taibi. "From static code analysis to visual models of microservice architecture." *Cluster Computing*27 (2024): 4145 – 4170.
49. Babar Ali, Muhammed Golec, S. Gill, Huaming Wu, Félix Cuadrado., and Steve Uhlig. "EdgeBus: Co-Simulation based resource management for heterogeneous mobile edge computing environments." *Internet Things*28 (2024): 101368.
50. Oyekunle Claudius Oyeniran, Adebunmi Okechukwu Adewusi, Adams Gbolahan Adeleke, Lucy Anthony Akwawa., and Chidimma Francisca Azubuko. "Microservices architecture in cloud-native applications: Design patterns and scalability." *Computer Science IT Research Journal*.
51. P. Thiruvassagam, Ieee Abhishek Chakraborty Member, A. Mathew, C. Siva, Ieee Ram Murthy Fellow., and Prabhu Kaliyam-mal. "Reliable Placement of Service Function Chains and Virtual Monitoring Functions With Minimal Cost in Softwarized 5G Networks." *IEEE Transactions on Network and Service Management*18 (2021): 1491–1507.
52. Shuai Zhao, Xiaotian Dai., and I. Bate. "DAG Scheduling and Analysis on Multi-core Systems by Modelling Parallelism and Dependency." *IEEE Transactions on Parallel and Distributed Systems*PP (2022): 1–1.
53. H. U. Adoga, and Dimitrios P. Pezaros. "Network Function Virtualization and Service Function Chaining Frameworks: A Comprehensive Review of Requirements, Objectives, Implementations, and Open Research Challenges." *Future Internet*14 (2022): 59.
54. Evelyn Borth, P. Lersch., and Annette Bieniusa. "Directed Acyclic Graph CRDTs." *Proceedings of the 12th Workshop on Principles and Practice of Consistency for Distributed Data*.
55. V. Eramo, E. Miucci, M. Ammar., and F. G. Lavacca. "An Approach for Service Function Chain Routing and Virtual Function Network Instance Migration in Network Function Virtualization Architectures." *IEEE/ACM Transactions on Networking*25 (2017): 2008–2025.
56. Xuanhao Chen, Yan Zhao, Guanfeng Liu, Rui Sun, Xiaofang Zhou., and Kai Zheng. "Efficient Similarity-Aware Influence Maximization in Geo-Social Network." *IEEE Transactions on Knowledge and Data Engineering*34 (2022): 4767–4780.
57. Ying Shi, Yuxia Zang, Tong Wang, Yiping Chen., and Chi Feng. "Dynamic Content Popularity Prediction Based on Spatial-Temporal Feature in Vehicular Networks." *Intell. Converged Networks*6 (2025): 326–341.
58. Yanxiang Jiang, Miaoli Ma, M. Bennis, F. Zheng., and X. You. "User Preference Learning-Based Edge Caching for Fog Radio Access Network." *IEEE Transactions on Communications*67 (2018): 1268–1283.
59. Gabriel A. Wainer 2009. *Discrete-Event Modeling and Simulation: A Practitioner's Approach*, USA: CRC Press, Inc.
60. X. Masip-Bruin, E. Marín-Tordera, S. Sánchez-López, Jordi García, A. Jukan, A. J. Ferrer, A. Queralt, Antonio Salis, A. Bartoli, Matija Cankar, C. Cordeiro, J. Jensen., and John Kennedy. "Managing the Cloud Continuum: Lessons Learnt from a Real Fog-to-Cloud Deployment." *Sensors (Basel, Switzerland)*21.
61. Benjamin Antunes, and David R. C. Hill. "Random Numbers for Machine Learning: A Comparative Study of Reproducibility and Energy Consumption." *Journal of Data Science and Intelligent Systems*.
62. Paulo Pereira, Carlos Melo, J. Araujo, J. Dantas, Vinícius Santos., and P. Maciel. "Availability model for edge-fog-cloud continuum: an evaluation of an end-to-end infrastructure of intelligent traffic management service." *The Journal of Supercomputing*78 (2021): 4421 – 4448.
63. James R. B. Bantock, B. Al-Hashimi., and G. Merrett. "Mitigating Interactive Performance Degradation From Mobile Device Thermal Throttling." *IEEE Embedded Systems Letters*13 (2020): 5–8.

64. Jinkun Zhang, Yuezhou Liu., and E. Yeh. "Optimal Congestion-aware Routing and Offloading in Collaborative Edge Computing." . *2022 20th International Symposium on Modeling and Optimization in Mobile, Ad hoc, and Wireless Networks (WiOpt)* (2022): 121–128.
65. L. A. Haibeh, M. Yagoub., and Abdallah Jarray. "A Survey on Mobile Edge Computing Infrastructure: Design, Resource Management, and Optimization Approaches." *IEEE Access*10 (2022): 27591–27610.
66. Isaac Ampratwum, and Amiya Nayak. "Radio link failure prediction in 5G networks using graph neural networks." . *2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)* (2025): 729–736.
67. Jonghyune Kim, V. Sridhara., and S. Bohacek. "Realistic mobility simulation of urban mesh networks." *Ad Hoc Networks*7 (2009): 411–430.
68. N. Khan, and F. Ahmad. "Modeling and simulation of an improved random direction mobility model for wireless networks using colored Petri nets." *SIMULATION*92 (2016): 323 – 336.
69. Bin Qiu, Ke Feng, Xian Li, Hailin Xiao., and Zhongshan Zhang. "Mobility-Aware User Association and Computation Offloading in Ultra-Dense Networks." *IEEE Transactions on Green Communications and Networking*9 (2025): 1524–1537.
70. Soule Issa Loutfi, Ibraheem Shaya, Ufuk Tureli, Ayman A. El-Saleh., and Waheeb Tashan. "An Overview of Mobility Awareness with Mobile Edge Computing over 6G Network: Challenges and Future Research Directions." . *Results in Engineering*.
71. Rui Li, Bertrand Decocq, A. Barros, Yining Fang., and Zhiguo Zeng. "Estimating 5G Network Service Resilience Against Short Timescale Traffic Variation." *IEEE Transactions on Network and Service Management*20 (2023): 2230–2243.
72. Abdelhak Kadouma, Ibrahim Afolabi, A. Shukla, Miloud Bagaa, Adlen Ksentini., and Mohammed S. Elmusrati. "A Unified Application Profile Model for Microservices-Based Applications in the Cloud-Edge Computing Continuum." . *2025 IEEE Conference on Network Function Virtualization and Software-Defined Networking (NFV-SDN)* (2025): 1–7.
73. L. Baresi, D. Mendonça, Martin Garriga, Sam Guinea., and G. Quattrocchi. "A Unified Model for the Mobile-Edge-Cloud Continuum." *ACM Transactions on Internet Technology (TOIT)*19 (2019): 1 – 21.
74. Kaihua Fu, Wei Zhang, Quan Chen, Deze Zeng., and M. Guo. "Adaptive Resource Efficient Microservice Deployment in Cloud-Edge Continuum." *IEEE Transactions on Parallel and Distributed Systems*33 (2022): 1825–1840.
75. Patrick Wendell, and Michael J. Freedman. 2011. "Going viral: flash crowds in an open CDN." In *Proceedings of the 2011 ACM SIGCOMM Conference on Internet Measurement Conference, IMC '11*, 549–558. New York, NY, USA: Association for Computing Machinery.
76. Shangguang Wang, Yan Guo, Ning Zhang, Peng Yang, Ao Zhou., and X. Shen. "Delay-Aware Microservice Coordination in Mobile Edge Computing: A Reinforcement Learning Approach." *IEEE Transactions on Mobile Computing*20 (2019): 939–951.
77. T. J. Schriber, and D. Brunner. "Inside discrete-event simulation software: how it works and why it matters." *Proceedings of the 2004 Winter Simulation Conference, 2004.1* (2004): –152.
78. O. Ullrich, and Daniel Lückerrath. "An Introduction to Discrete-Event Modeling and Simulation." *Simul. Notes Eur.*27 (2017): 9–16.
79. W. Higashino, Miriam A. M. Capretz., and L. Bittencourt. "CEPSim: Modelling and simulation of Complex Event Processing systems in cloud environments." *Future Gener. Comput. Syst.*65 (2016): 122–139.
80. Feng Zhu, Yiping Yao, Wenjie Tang., and Jun Tang. "A hierarchical composite framework of parallel discrete event simulation for modelling complex adaptive systems." *Simul. Model. Pract. Theory*77 (2017): 141–156.
- 81.
82. Jinbo Xiong, Xiuhua Chen, Q. Yang, Lei Chen., and Zhiqiang Yao. "A Task-Oriented User Selection Incentive Mechanism in Edge-Aided Mobile Crowdsensing." *IEEE Transactions on Network Science and Engineering*7 (2019): 2347–2360.
83. D. Wu, Jia Liu., and Zhigang Yang. "Bilateral Satisfaction Aware Participant Selection With MEC for Mobile Crowd Sensing." *IEEE Access*8 (2020): 48110–48122.
84. Ting Li, Zhijin Qiu, Lijuan Cao, Dazhao Cheng, Weichao Wang, Xinghua Shi., and Yu Wang. "Privacy-Preserving Participant Grouping for Mobile Social Sensing Over Edge Clouds." *IEEE Transactions on Network Science and Engineering*8 (2020): 865–880.
85. Chih-Hang Wang, Jian-Jhih Kuo, De-Nian Yang., and Wen-Tsuen Chen. "Collaborative Social Internet of Things in Mobile Edge Networks." *IEEE Internet of Things Journal*7 (2020): 11473–11491.
86. Xiaomei Huang, Junhao Zhao, Jianyao Li, Xiaonan He, Zhiheng Zhou, Aiwen Jiang., and Neal N. Xiong. "Edge-Enabled Context-Aware Group Recommendation in Sensor-Cloud Networks: Collaboration Between Large-Scale and Lightweight Models." *IEEE Network*40 (2026): 104–112.
87. Gayan Dihantha Kuruppu Kuruppu Appuhamilage, Maqbool Hussain, M. Zaman., and Wajahat Ali Khan. "A health digital twin framework for discrete event simulation based optimised critical care workflows." *NPJ Digital Medicine*8.
88. Benjamin Reidys, Pantea Zardoshti, Íñigo Goiri, Celine Irvine, Daniel S. Berger, Haoran Ma, K. Arya, Eli Cortez, Taylor Stark, Eugene Bak, M. Iyigun, Stanko Novaković, Lisa Hsu, Karel Trueba, Abhisek Pan, Chetan Bansal, Saravan Rajmohan, Jian Huang., and Ricardo Bianchini. "Coach: Exploiting Temporal Patterns for All-Resource Oversubscription in Cloud Platforms." . *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*.
89. Ziliang Qiu, Wei Deng, Juzheng Huang, Xuezhi Liu, Binhua Ren., and Siyuan Qin. "Cloud platform resource capacity management and scheduling optimization based on data graph." . *Proceedings of the 2025 International Conference on Big Data, Communication Technology and Computer Applications*.

90. Muhammad Zakarya, and Lee Gillam. "Modelling resource heterogeneities in cloud simulations and quantifying their accuracy." *Simul. Model. Pract. Theory*94 (2019): 43–65.
91. Yuhan Yi, Guanglin Zhang,, and Hai Jiang. "Mobile Edge Computing Networks: Online Low-Latency and Fresh Service Provisioning." *IEEE Transactions on Communications*73 (2024): 11463–11479.
92. Bulent Soykan, and G. Rabadi. "Optimizing Job Shop Scheduling Problem Through Deep Reinforcement Learning and Discrete Event Simulation." . *2024 Winter Simulation Conference (WSC)* (2024): 2607–2618.
93. G. Santos, Diego de Freitas Bezerra, É. Rocha, Leylane Ferreira, A. Moreira, G. Gonçalves, Maria Valéria Marquezini, Ákos Recse, Amardeep Mehta, J. Kelner, D. Sadok,, and P. Endo. "Service Function Chain Placement in Distributed Scenarios: A Systematic Review." *Journal of Network and Systems Management*30.
94. Xiang He, Hanchuan Xu, Xiaofei Xu, Yin Chen,, and Zhongjie Wang. "An Efficient Algorithm for Microservice Placement in Cloud-Edge Collaborative Computing Environment." *IEEE Transactions on Services Computing*17 (2024): 1983–1997.
95. Jiangkai Peng, Bo Fan, Zhenghong Tu, Wei Zhang,, and Wenxin Liu. "Distributed Periodic Event-Triggered Optimal Control of DC Microgrids Based on Virtual Incremental Cost." *IEEE/CAA Journal of Automatica Sinica*9 (2022): 624–634.
96. P. Soumplis, Georgios Kontos, Panagiotis C. Kokkinos, A. Kretsis, Sergio Barrachina-Muñoz, Rasoul Nikbakht, J. Baranda, Miquel Payaró, J. Manges-Bafalluy,, and Emmanuel Varvarigos. "Performance Optimization Across the Edge-Cloud Continuum: A Multi-agent Rollout Approach for Cloud-Native Application Workload Placement." *SN Computer Science*5.
97. Masoud Taghavian, Yassine Hadjadj-Aoul, Géraldine Texier, N. Huin,, and Philippe Bertin. "An Approach to Network Service Placement Reconciling Optimality and Scalability." *IEEE Transactions on Network and Service Management*20 (2023): 2218–2229.
98. Omar Alhussein, Phu Thinh Do, Qiang Ye, Junling Li, Weisen Shi, Weihua Zhuang, Xuemin Shen, Xu Li,, and Jaya Rao. "A Virtual Network Customization Framework for Multicast Services in NFV-Enabled Core Networks." *IEEE Journal on Selected Areas in Communications* 38, no. 6 (2020): 1025–1039.
99. Yue Peng, Jiwen Xia, Dafeng Liu, Miao Liu, Long Xiao,, and Benyun Shi. "Unifying topological structure and self-attention mechanism for node classification in directed networks." *Scientific Reports*15.
100. Longfei Zhang, Yang Feng, Rongxiao Wang, Yue Xu, Naifu Xu, Zeyi Liu,, and Hang Du. "Efficient experience replay architecture for offline reinforcement learning." . *Robotic Intelligence and Automation*.
101. Shuran Sheng, Peng Chen, Zhimin Chen, Lenan Wu,, and Yu Yao. "Deep Reinforcement Learning-Based Task Scheduling in IoT Edge Computing." *Sensors (Basel, Switzerland)*21.
102. Jiagang Liu, Yun Mi,, and Xinyu Zhang. "Task Graph offloading via Deep Reinforcement Learning in Mobile Edge Computing." *Future Gener. Comput. Syst.*158 (2023): 545–555.
103. Rafael Figueiredo Prudencio, M. Maximo,, and E. Colombini. "A Survey on Offline Reinforcement Learning: Taxonomy, Review, and Open Problems." *IEEE Transactions on Neural Networks and Learning Systems*35 (2022): 10237–10257.
104. P. Mannion, Sam Devlin, J. Duggan,, and E. Howley. "Reward shaping for knowledge-based multi-objective multi-agent reinforcement learning." *The Knowledge Engineering Review*33.
105. Ni Mu, Yao Luan,, and Qing-Shan Jia. "Preference-Based Multi-Objective Reinforcement Learning." *IEEE Transactions on Automation Science and Engineering*22 (2025): 18737–18749.
106. Charles Reiss, Alexey Tumanov, Gregory R. Ganger, Randy H. Katz,, and Michael A. Kozuch. 2012. "Heterogeneity and dynamicity of clouds at scale: Google trace analysis." In *Proceedings of the Third ACM Symposium on Cloud Computing, SoCC '12*, New York, NY, USA: Association for Computing Machinery.
107. Theophilus Benson, Aditya Akella,, and David A. Maltz. 2010. "Network traffic characteristics of data centers in the wild." In *Proceedings of the 10th ACM SIGCOMM Conference on Internet Measurement, IMC '10*, 267–280. New York, NY, USA: Association for Computing Machinery.
108. Bianca Schroeder, and Garth A Gibson. "Understanding failures in petascale computers." *Journal of Physics: Conference Series* 78, no. 1 (2007): 012022.
109. Philipp Leitner, and Jürgen Cito. "Patterns in the Chaos—A Study of Performance Variation and Predictability in Public IaaS Clouds." *ACM Trans. Internet Technol.* 16, no. 3.
110. Eli Cortez, Anand Bonde, Alexandre Muzio, Mark Russinovich, Marcus Fontoura,, and Ricardo Bianchini. 2017. "Resource Central: Understanding and Predicting Workloads for Improved Resource Management in Large Cloud Platforms." In *Proceedings of the 26th Symposium on Operating Systems Principles, SOSP '17*, 153–167. New York, NY, USA: Association for Computing Machinery.
111. L. Breslau, Pei Cao, Li Fan, G. Phillips,, and S. Shenker. 1999. "Web caching and Zipf-like distributions: evidence and implications." In *IEEE INFOCOM '99. Conference on Computer Communications. Proceedings. Eighteenth Annual Joint Conference of the IEEE Computer and Communications Societies. The Future is Now (Cat. No.99CH36320)*, Vol 1, 126–134 vol.1.
112. Meeyoung Cha, Haewoon Kwak, Pablo Rodriguez, Yong-Yeol Ahn,, and Sue Moon. 2007. "I tube, you tube, everybody tubes: analyzing the world's largest user generated content video system." In *Proceedings of the 7th ACM SIGCOMM Conference on Internet Measurement, IMC '07*, 1–14. New York, NY, USA: Association for Computing Machinery.
113. Alexandru Uta, Alexandru Custura, Dmitry Duplyakin, Ivo Jimenez, Jan Rellermeyer, Carlos Maltzahn, Robert Ricci,, and Alexandru Iosup. 2020. "Is big data performance reproducible in modern cloud networks?." In *Proceedings of the 17th Usenix Conference on Networked Systems Design and Implementation, NSDI'20*, 513–528. USA: USENIX Association.
114. Tracy Camp, Jeff Boleng,, and Vanessa Davies. "A survey of mobility models for ad hoc network research." *Wireless Communications and Mobile Computing* 2, no. 5 (2002): 483–502.
115. Theodore S. Rappaport 2024. *Wireless Communications: Principles and Practice*, Cambridge University Press.

116. Vern Paxson, and Sally Floyd. “Wide-area traffic: the failure of Poisson modeling.” *SIGCOMM Comput. Commun. Rev.* 24, no. 4 (1994): 257–268.
117. Junxian Huang, Feng Qian, Alexandre Gerber, Z. Morley Mao, Subhabrata Sen., and Oliver Spatscheck. 2012. “A close examination of performance and power characteristics of 4G LTE networks.” In *Proceedings of the 10th International Conference on Mobile Systems, Applications, and Services*, MobiSys ’12, 225–238. New York, NY, USA: Association for Computing Machinery.
118. Martin F. Arlitt, and Carey L. Williamson. “Web server workload characterization: the search for invariants.” *SIGMETRICS Perform. Eval. Rev.* 24, no. 1 (1996): 126–137.
119. Tarik Taleb, Konstantinos Samdanis, Badr Mada, Hannu Flinck, Sunny Dutta., and Dario Sabella. “On Multi-Access Edge Computing: A Survey of the Emerging 5G Network Edge Cloud Architecture and Orchestration.” *IEEE Communications Surveys Tutorials* 19, no. 3 (2017): 1657–1681.
120. Christian Bettstetter 2002. “On the minimum node degree and connectivity of a wireless multihop network.” In *Proceedings of the 3rd ACM International Symposium on Mobile Ad Hoc Networking & Computing*, MobiHoc ’02, 80–91. New York, NY, USA: Association for Computing Machinery.
121. Michalis Faloutsos, Petros Faloutsos., and Christos Faloutsos. 1999. “On power-law relationships of the Internet topology.” In *Proceedings of the Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication*, SIGCOMM ’99, 251–262. New York, NY, USA: Association for Computing Machinery.
122. K.L. Calvert, M.B. Doar., and E.W. Zegura. “Modeling Internet topology.” *IEEE Communications Magazine* 35, no. 6 (1997): 160–163.
123. Alberto Medina, Anukool Lakhina, Ibrahim Matta., and John Byers. 2001. “BRITE: An Approach to Universal Topology Generation.” *MASCOTS ’01*, 346. USA: IEEE Computer Society.
124. K.-I. Goh, B. Kahng., and D. Kim. “Universal Behavior of Load Distribution in Scale-Free Networks.” *Phys. Rev. Lett.* 87 (2001): 278701.
125. Athina Markopoulou, Gianluca Iannaccone, Supratik Bhattacharyya, Chen-Nee Chuah., and Christophe Diot. 2004. “Characterization of Failures in an IP Backbone.” In *Proceedings IEEE INFOCOM 2004*, Vol 4, 2307–2317.

APPENDIX

A | List of Available Statistical Distributions

A fundamental design principle of ECCO-GENERATOR is its configuration flexibility. While the framework suggests specific statistical distributions grounded in Computing Continuum and reliability literature to ensure maximum realism, researchers are not locked into these defaults. Every event, its temporality, and its specific behavioral parameters are fully customizable via a YAML configuration file, allowing the substitution of distributions (e.g., swapping a Poisson process for a deterministic interval) without modifying the underlying source code. A complete set of predefined statistical distributions is available to the user (Table A1).

B | YAML Configuration File Schema for Scenario Definition

This appendix describes the exact format of the YAML configuration file used to define an ECCOS simulation scenario. This file specifies the scenario structure and stochastic behavior consumed by the simulator, including the infrastructure, applications, users, global spawners, and event definitions. The configuration filename is defined in the simulator’s `constants.py` file, although it can also be provided explicitly through the command line when launching a simulation. By default, ECCOS expects this file to be named `scenario_config.yaml`.

To provide fine-grained control over experimental reproducibility, the scenario configuration YAML file supports a dedicated `seeds` block. First, users may define either a single master seed, which automatically derives deterministic, sequential sub-seeds across all simulation domains. Alternatively, users may explicitly specify independent integer seeds for individual conceptual domains (`graph`, `app`, `user`, and `event`). When explicit domain seeds are provided, they override the master derivation for that specific scope.

Listing 1 YAML configuration for stochastic experimental reproducibility through seeds definition.

```
seeds:
  master: 42
  graph: 42
  app: 43
  user: 44
  event: 45
```

TABLE A1 | Statistical distributions supported by the YAML configuration, inspired by `numpy.random`.

Distribution	Parameters (Type)	YAML Example
Constant (Numeric Literal)	None; direct constant value	<code>distribution: 100</code>
Normal (Gaussian)	<code>loc (float)</code> <code>scale (float)</code>	<code>distribution:</code> <code> type: normal</code> <code> loc: 10.0</code> <code> scale: 2.5</code>
Uniform	<code>low (float)</code> <code>high (float)</code>	<code>distribution:</code> <code> type: uniform</code> <code> low: 0.0</code> <code> high: 1.0</code>
Exponential	<code>scale (float)</code>	<code>distribution:</code> <code> type: exponential</code> <code> scale: 5.0</code>
Poisson	<code>lam (float)</code>	<code>distribution:</code> <code> type: poisson</code> <code> lam: 4.0</code>
Pareto	<code>a (float)</code> <code>size (string/expression)</code>	<code>distribution:</code> <code> type: pareto</code> <code> a: 1.5</code> <code> size: "{num_nodes}"</code>
Integers	<code>low (integer)</code> <code>high (integer)</code>	<code>distribution:</code> <code> type: integers</code> <code> low: 1</code> <code> high: 10</code>
Choice (Set Selection)	<code>a (string/expression)</code>	<code>distribution:</code> <code> type: choice</code> <code> a: "{active_edges}"</code>
Binomial	<code>n (integer)</code> <code>p (float)</code>	<code>distribution:</code> <code> type: binomial</code> <code> n: 10</code> <code> p: 0.5</code>
Gamma	<code>shape (float)</code> <code>scale (float)</code>	<code>distribution:</code> <code> type: gamma</code> <code> shape: 2.0</code> <code> scale: 1.0</code>
Beta	<code>a (float)</code> <code>b (float)</code>	<code>distribution:</code> <code> type: beta</code> <code> a: 0.5</code> <code> b: 0.5</code>
Lognormal	<code>mean (float)</code> <code>sigma (float)</code>	<code>distribution:</code> <code> type: lognormal</code> <code> mean: 0.0</code> <code> sigma: 1.0</code>
Weibull	<code>a (float)</code>	<code>distribution:</code> <code> type: weibull</code> <code> a: 1.5</code> <code> scale: 10.0</code>
Geometric	<code>p (float)</code>	<code>distribution:</code> <code> type: geometric</code> <code> p: 0.35</code>
Dirichlet	<code>alpha (list of floats)</code>	<code>distribution:</code> <code> type: dirichlet</code> <code> alpha: [10.0, 5.0, 2.0]</code>
Multinomial	<code>n (integer)</code> <code>pvals (list of floats)</code>	<code>distribution:</code> <code> type: multinomial</code> <code> n: 20</code> <code> pvals: [1.0, 1.0, 1.0]</code>
Triangular	<code>left (float)</code> <code>mode (float)</code> <code>right (float)</code>	<code>distribution:</code> <code> type: triangular</code> <code> left: 0.0</code> <code> mode: 0.5</code> <code> right: 1.0</code>
Rayleigh	<code>scale (float)</code>	<code>distribution:</code> <code> type: rayleigh</code> <code> scale: 1.0</code>
Standard Cauchy	None	<code>distribution:</code> <code> type: standard_cauchy</code>
Random Float [0.0, 1.0)	None	<code>distribution:</code> <code> type: random</code>

B.1 | Infrastructure Definition

The infrastructure definition in Ecco-Architect is driven by the number of nodes (`num_nodes`), the topological model for the generation of the graph structure (`model`), and the mapping strategy to map graph nodes to the computing continuum layers (`model.layer.mode`): cloud, fog and edge. The topological model and the mapping of layers requires of additional parameters in some cases.

Listing 2 YAML configuration for infrastructure definition.

```
infrastructure:
  num_nodes: 50
  model:
    name: scale_free          # Values (erdos_renyi | scale_free | spatial | fat_tree |
                              # tree | multi_tier)
    m: 2                     # For scale_free. Number of edges to attach from a new node
                              # to existing nodes
    radius: 0.1              # For spatial. Distance radius for connecting nodes in the
                              # geometric graph
    branching_r: 2           # For tree/fat_tree. Branching factor of the tree (number of
                              # children per node)
    branching_h: 4           # For tree/fat_tree. Height or depth of the tree
    cloud:
      percentage: 2          # Percentage of total nodes assigned to the cloud layer
    fog:
      percentage: 10         # Percentage of total nodes assigned to the fog layer
      m: 2                   # Number of edges to attach per new fog node in its scale-
                              # free topology (default: 2)
    edge:
      radius: 0.1           # Connection distance radius for the spatial edge layer
                              # (default: 0.1)
    layer:
      mode: centrality_based_layer # Values (centrality_based_layer | depth_based_layer)
      thresholds:
        cloud_min: 0.1       # For centrality_based_layer Minimum centrality score (0.0
                              # to 1.0) to be considered 'cloud'
        fog_min: 0.02        # For centrality_based_layer Minimum centrality score to be
                              # considered 'fog' (nodes below this become 'edge')
        cloud_max: 0         # For depth_based_layer. Only applies to 'tree'/'fat_tree'
                              # models. Maximum depth level to be considered 'cloud'
                              # (usually 0 for root)
        fog_max: 2           # For depth_based_layer. Only applies to 'tree'/'fat_tree'
                              # models. Maximum depth level to be considered 'fog' (nodes
                              # deeper than this become 'edge')
```

ECCOS characterizes nodes and edges with unlimited user-defined attributes (`node.attributes`, `edge.attributes`) to describe resources, operational properties...The configuration must define how its initial values are assigned across the generated topology (`node.attributes.<name>.mode`) and general bounds values (`node.attributes.<name>.min/max`). Additionally, the attribute distribution allows to define the statistical model to generate the values. In this block, specific layer definitios can overwrite the general definition, allowing to express a specific configuration for the nodes/edges of a given layer (`node.attributes.<name>.distribution.cloud/fog/edge`).

Listing 3 YAML configuration for the attributes definition of the infrastructure entities.

```
infrastructure:
  node:
    attributes:
      cpu:
        mode: centrality_based # Values (uniform | centrality_based | layered)
        centrality_type: direct_proportional # For centrality_based. Values (direct_proportional
                                                # | inverted proportional)
        min: 4.0
        max: 16.0
        distribution:
          type: pareto
          a: 1.16
          cloud: # Overrides distribution and max for cloud nodes.
            type: uniform
            low: 128.0
            high: 512.0
```

```

        max: 1024.0
ram:
  mode: layered
  min: 1.0
  max: 1024.0
  distribution:
    cloud:
      type: uniform
      low: 256.0
      high: 512.0
      min: 200.0
      max: 1024.0
    fog:
      type: normal
      loc: 32.0
      scale: 8.0
      min: 8.0
      max: 64.0
  edge:
    type: uniform
    low: 2.0
    high: 8.0
# General bounds
# Overwrites values
# <-- Overwrites values
# No explicit min/max values, it inherits the
# global 1.0 and 1024.0 bounds

```

B.2 | Infrastructure Events

Events to disable nodes and edges (`disable_node`, `disable_edge`) are configured in terms of their frequency and their impact. Frequency is modeled through the use of a statistical distribution to define their inter-arrival time. The impact of the disable events is measured in terms of the MTTR. Thus, both events have the block `distribution_to_enable_node` in their impact block, also modeled with a statistical distribution.

Listing 4 YAML configuration for disable events of infrastructure elements

```

infrastructure:
  node:
    actions:
      disable_node:
        frequency:
          type: exponential
          scale: 300
        impact:
          distribution_to_enable_node:
            type: uniform
            low: 0
            high: 100
  edge:
    actions:
      disable_edge:
        frequency:
          type: exponential
          scale: 300
        impact:
          distribution_to_enable_edge:
            type: uniform
            low: 0
            high: 100

```

Degradation events such as `degrade_node` and `congest_edge` are also modeled through a statistical distribution for their inter-arrival times (`frequency`). But in this case, the impact is not only configured with the MTTR distribution (`impact.distribution_to_restore_node/distribution_to_clear_edge`), but also by the list of attributes that are degraded or multiplied (`impact.target_attributes/target_attributes_loss/target_attributtes_multiplier`), and the statistical distribution that govern those losses/multipliers (`impact.p_loss_dist/multiplier_dist`).

Listing 5 YAML configuration for degrade events of infrastructure elements

```

infrastructure:
  node:
    actions:

```

```

degrade_node:
  frequency:
    type: exponential
    scale: 150
  impact:
    target_attributes: ["ram", "cpu"]
    p_loss_dist:
      type: beta
      a: 2
      b: 5
    distribution_to_restore_node:
      type: exponential
      scale: 50
edge:
  actions:
    congest_edge:
      frequency:
        type: exponential
        scale: 150
      impact:
        target_attributes_loss: ["bandwidth"]
        target_attributes_multiplier: ["delay"]
        p_loss_dist:
          type: beta
          a: 2
          b: 5
        multiplier_dist:
          type: lognormal
          mean: 0.5
          sigma: 0.2
        distribution_to_clear_edge:
          type: exponential
          scale: 50

```

B.3 | User Definition

Users are modelled by the initial number of users (`num_users`), the statistical distribution to determine their request ratios (`request_ratio`), the spatial size of the simulated area where the users can move around (`spatial_region.width/height`), the streets of this simulated area (`spatial_region.num_vertical_streets/num_horizontal_streets`), and the model for the initial position of the users (`spatial_distribution`).

Listing 6 YAML configuration for users creation.

```

user:
  num_users: 2
  request_ratio:
    type: exponential
    scale: 5
  spatial_distribution:
    model: thomas_cluster          # Values (random_uniform | thomas_cluster)
    hotspots: 5                   # only for thomas_cluster
    spread: 0.05                  # only for thomas_cluster
  spatial_region:
    width: 2.0
    height: 2.0
    num_vertical_streets: 10
    num_horizontal_streets: 10

```

Additionally, user mobility patterns are configured through the mobility model(`model`), the direction-change or turning probabilities (`turn_probabilities`), the movement speed (`speed`), and the distance that defines the coverage range to the connection node for each user (`coverage_radius`).

Listing 7 YAML configuration for the definition of the users mobility mode.

```

user:
  mobility:
    model: random                 # Values (static | random | manhattan)
    turn_probabilities:

```

```

    straight: 0.33
    left: 0.33           # Only for manhattan
    right: 0.34          # Only for manhattan
  speed:
    type: uniform
    low: 0.005
    high: 0.02
  coverage_radius:
    type: normal
    mean: 0.2
    sigma: 0.02

```

B.4 | User Events

The user removal event (`remove_user`) is modeled through its frequency and the parametrization of the corresponding statistical distribution, and therefore does not require an impact configuration. Similarly, user mobility across the two-dimensional simulation plane is modeled by the user movement event (`move_user`), together with its corresponding frequency distribution. Its impact is determined by the mobility configuration established during user definition and creation.

Listing 8 YAML configuration for basic user events.

```

user:
  actions:
    remove_user:
      distribution:
        type: exponential
        scale: 300
    move_user:
      frequency:
        type: exponential
        scale: 120

```

Temporary user disconnections are triggered by the suspend-user event (`suspend_user`). In addition, an automatic reconnection event is inserted into the event queue to simulate device recovery. Both the frequency of user disconnections and the time required to restore connectivity (`distribution_to_resume_user`) are modeled through their corresponding statistical distributions.

Listing 9 YAML configuration for user disconnection event.

```

user:
  actions:
    suspend_user:
      frequency:
        type: exponential
        scale: 150
    impact:
      distribution_to_resume_user:
        type: lognormal
        mean: 1.0
        sigma: 0.5

```

Temporary changes in users' request ratios are modeled through the `change_request_ratio` event and its inter-arrival frequency distribution. The event impact is configured through the probability of producing either a burst, which increases requests, or a valley, which decreases them (`increase_probability`), together with the multiplier magnitude of the request-ratio modification (`multiplier`) and the time required to restore the initial ratio (`restore_ratio`).

Listing 10 YAML configuration for activity changes user event.

```

user:
  actions:
    change_request_ratio:
      frequency:
        type: exponential
        scale: 90
    impact:
      increase_probability: 0.5
      multiplier:

```

```

    type: uniform
    low: 1.5
    high: 3.0
  restore_ratio:
    type: lognormal
    mean: 1.0
    sigma: 0.5

```

B.5 | Application Definition

The simulator must know the number of applications to generate at the beginning of the simulation (`num_apps`). At the same time, the configuration defines the range of services that may compose each application (`services.num_services_range`) and how these applications are handled during placement (`architecture`): either as a single monolithic block that must be deployed on one node (`monolithic`), or as separable microservices that can be placed across different nodes (`microservices`). For microservice-based applications, the service composition can be configured (`topology_model`) either as a Service Function Chain (SFC), which imposes a strict sequential order among services, or as a Directed Acyclic Graph (DAG). In the latter case, the configuration must specify the graph-generation parameters, including the initial number of nodes (`initial_nodes`) and the number of edges attached per node (`edges_per_node`).

Listing 11 YAML configuration for application creation.

```

app:
  num_apps: 20
  architecture: monolithic           # Values (monolithic | microservice)
  services:
    num_services_range: [2, 6]
    topology_model: directed_scale_free # Values (sfc | directed_scale_free)
    scale_free_settings:               # Only for directed_scale_free
      initial_nodes: 2
      edges_per_node: 2

```

The simulator defines the network resource consumptions as attributes associated to the application (`app.services.network.<attribute_name>`). On contrary, the computational resource consumptions are associated to services through the use of profile definitions (`app.services.profiles.<profile_name>.<attribute_name>`). In the later case, the profiles are selected uniformly random through the assigned probability (`prob`).

Listing 12 YAML configuration for resource consumptions and profiles definition of the services.

```

app:
  services:
    network:
      delay:
        distribution:
          type: uniform
          low: 10.0
          high: 50.0
      bandwidth:
        distribution:
          type: uniform
          low: 10.0
          high: 100.0
    profiles:
      iot:
        prob: 0.6
        attributes:
          cpu:
            distribution:
              type: lognormal
              mean: 0.1
              sigma: 0.5
          ram:
            distribution:
              type: lognormal
              mean: 0.1
              sigma: 0.5
      video:

```

```

prob: 0.4
attributes:
  cpu:
    distribution:
      type: lognormal
      mean: 2.0
      sigma: 1.0
  ram:
    distribution:
      type: lognormal
      mean: 3.0
      sigma: 1.5

```

Finally, applications must define their popularity (popularity) through a statistical distribution. The popularity is the probability with which applications are requested by newly created users or by the initial users at the beginning of the simulation. The configuration also allows a subset of applications to be marked as local (`local_app_ratio`), increasing their popularity within their corresponding influence radius (`local_radius_influence`). To avoid creating applications without immediate demand, the simulator also defines the number of new users to be generated and associated with each newly created application (`num_new_users`).

Listing 13 YAML configuration for application popularity lifecycle.

```

app:
  popularity:
    distribution:
      type: zipf
      a: 1.2
    local_app_ratio: 0.3
    local_radius_influence: 0.15
  num_new_users:
    type: integers
    low: 1
    high: 6

```

B.6 | Application Events

Application deletion event (`remove_app`) is simply modelled with its frequency distribution.

Listing 14 YAML configuration for removal application event.

```

app:
  actions:
    remove_app:
      frequency:
        type: uniform
        low: 0
        high: 300

```

The events to change the application workloads (`update_app_footprint`) and network usages (`update_app_network`) are identically modeled through their frequency, and the impact based on the distribution for the standard deviation of the applied noise (`sigma_footprint`).

Listing 15 YAML configuration for application workload changes events.

```

app:
  actions:
    update_app_footprint:
      frequency:
        type: exponential
        scale: 100
      impact:
        sigma_footprint:
          type: uniform
          low: 0.05
          high: 0.15
    update_app_network:
      frequency:
        type: exponential
        scale: 150

```

```

    impact:
      sigma_net:
        type: uniform
        low: 0.05
        high: 0.15

```

The event that changes the topological structure of applications in microservice-defined architectures is `update_app_topology`. As in the other cases, this event is modeled by the individual frequency of each entity and by its impact, which is measured in terms of the minimum and maximum number of services that remain after the application update (`l_min` and `l_max_global`).

Listing 16 YAML configuration for application lifecycle event.

```

app:
  actions:
    update_app_topology:
      frequency:
        type: exponential
        scale: 120
      impact:
        l_min:
          type: integers
          low: 2
          high: 3
        l_max_global:
          type: integers
          low: 6
          high: 7

```

Changes in application popularity, restricted to newly created users, are defined through the `surge_popularity` and `drop_popularity` events, which respectively increase or decrease an application's popularity. This popularity change can be transient or permanent, as modeled by the statistical distribution in `transient_prob`; for transient changes, the duration of the updated popularity is defined by `duration_dist`.

Listing 17 YAML configuration for application popularity events.

```

app:
  actions:
    surge_popularity:           # or drop_popularity:
      frequency:
        type: exponential
        scale: 200
      impact:
        transient_prob:
          type: uniform
          low: 0.7
          high: 0.7
        duration_dist:
          type: weibull
          a: 1.5
          scale: 10.0

```

Changes in the location of the hot spot for localized applications can be triggered through the `geo_demand_shift` event, which moves the hot spot either by a random jump or by an incremental movement, according to the `jump_prob` parameter. In the case of drift-based movement of the hot spot, the statistical distribution in `drift_dist` determines the shift from the current coordinates.

Listing 18 YAML configuration for hot spot changes for localized applications.

```

app:
  actions:
    geo_demand_shift:
      frequency:
        type: exponential
        scale: 50
      impact:
        jump_prob:
          type: uniform
          low: 0.2

```

```

    high: 0.2
    drift_dist:
      type: normal
      mean: 0.0
      sigma: 0.05

```

B.7 | Global Spawner Events

Creating new users and applications are global simulation events that represents the external arrival of a new entities into the scenario. Therefore, these events are defined in the global configuration entries `global_spawner.actions.new_user` and `global_spawner.actions.new_app`, rather than inside the respective action blocks: `user.actions` or `app.actions`.

Unlike domain-specific events where the target entity is implicit, the global orchestration block `global_spawner` requires `type_object` to dynamically route events to their appropriate domain managers (the `UserSet` or `ApplicationSet` catalogs). Simultaneously, `action_type` maps the configuration event directly to the specific internal handler method via runtime reflection; this separates declarative configuration naming from code implementation. This separation for the specific case of global orchestration enables complex, cross-domain user-defined composite workflows.

The `type_object` attribute also supports the special value `type_object:global` for user-defined composed events (Section B.8).

Listing 19 YAML configuration for global entity creation events

```

global_spawner:
  actions:
    new_user:
      type_object: user           # Values: (user | app | global)
      action_type: new_user      # Values: (new_user | new_app)
      frequency:
        type: exponential
        scale: 30
    new_app:
      type_object: app
      action_type: new_app
      frequency:
        type: exponential
        scale: 3600

```

B.8 | User Composition of Encapsulated Events

An encapsulated event is defined, like other events, as an element of the `actions` attribute, using a user-defined name (`actions.<event_name>`) and its corresponding inter-arrival frequency (`frequency`). In addition, its impact specification includes a list of events under the `impact.composed_of` attribute. Because of the generic nature of the composition model, each event in this list must explicitly declare the `type_object` and `action_type` attributes. The `type_object` attribute indicates the entity type that handles the event, whereas `action_type` specifies the exact base action to trigger.

Table B2 includes an exhaustive enumeration of all the `action_type` alternatives. It is important to note that this table includes more actions than the set presented as base configurable actions in the main text. This is intentional: some entries correspond to derived actions that are internally triggered by other base events, but may still be useful when defining composed events. For example, `remove_user_by_requested_app` is derived from `remove_app`, since removing an application also requires removing all users requesting it. Similarly, `revive_edge` is automatically triggered after `disable_edge` according to the configured recovery process. These derived actions are typically less useful as standalone base-event configurations, but they provide additional flexibility when constructing user-defined composed actions.

Listing 20 YAML configuration of a combined move-increase activity user event.

```

user:
  actions:
    move_and_increase_request:
      frequency:
        type: exponential
        scale: 600
      impact:
        composed_of:
          # The user moves and [...]

```

TABLE B2 | Taxonomy of composite action types (`action_type`) categorized by simulation entity domain (`type_object`).

Domain (<code>type_object</code>)	Action (<code>action_type</code>)	Primary Simulation Impact	Operational Context
user	<code>move_user</code>	Displaces (X, Y) coordinates based on mobility models.	Mobile user tracking, handover tests.
	<code>change_request_ratio</code>	Modifies workload generation frequency multiplier.	Flash crowds, local demand spikes.
	<code>suspend_user</code>	Temporarily halts user session and traffic generation.	Network outages, disconnections.
	<code>new_user</code>	Instantiates or reactivates a user entity dynamically.	Diurnal workload scaling, arrivals.
	<code>remove_user</code>	Permanently deletes user instance and pending events.	Session termination, cleanup.
	<code>remove_user_by_requested_app</code>	Batch removal of users linked to a specific application.	Cascading app decommissioning.
app	<code>surge_popularity</code>	Triggers transient upward surge in request volume.	Viral campaigns, sudden traffic peaks.
	<code>drop_popularity</code>	Induces transient decrease in request popularity.	Off-peak transitions, user decay.
	<code>geo_demand_shift</code>	Relocates app geographic focus via jump or drift.	Regional demand migration.
	<code>update_app_footprint</code>	Modifies service CPU, RAM, and storage requirements.	Vertical scaling, heavy processing.
	<code>update_app_network</code>	Alters bandwidth requirements and latency constraints.	QoS policy updates, SLA adjustments.
	<code>update_app_topology</code> <code>new_app / remove_app</code>	Mutates service graph within $[l_{\min}, l_{\max}]$ bounds. Dynamically spawns or destroys application graphs.	Horizontal architectural adaptation. Lifecycle orchestration, multi-tenancy.
graph_node	<code>disable_node</code>	Disconnects server, triggering service migrations.	Hardware failures, power outages.
	<code>degrade_node</code>	Reduces available server CPU/RAM by loss percentage.	Resource exhaustion, overload.
	<code>revive_node / restore_node</code>	Restores server to full operational capacity.	Maintenance completion, recovery.
graph_edge	<code>disable_edge</code>	Severs link, forcing routing and shortest-path recalculation.	Fiber cuts, wireless link loss.
	<code>congest_edge</code>	Degrades link bandwidth and inflates packet loss/latency.	Network congestion, DDoS attacks.
	<code>revive_edge / clear_edge</code>	Restores network link to standard operating parameters.	Link restoration, congestion clearing.

```

- action_type: move_user          # Values listed in Table B2
  type_object: user               # Values (user | app | graph_node | graph_edge)
  impact_params:
    target_resolution:
      mode: self
- action_type: change_request_ratio # [...] the user increase request ratio
  type_object: user
  impact_params:
    target_resolution:
      mode: self
    multiplier:
      type: uniform
      low: 1.5
      high: 3.0

```

Each event listed in `composed_of` must also model its own impact, in the same way as when that event is configured independently. In encapsulated events, this impact configuration is placed under the `impact_params` attribute of each composed sub-action. The expected subattributes are therefore the same as those required by the corresponding standalone event; only their location in the YAML hierarchy changes. Additionally, the new parameter `target_resolution` is used to select the entities to apply the event.

This `target_resolution` attribute supports the following modes: (a) `self`, the default behavior. The sub-action is applied strictly to the same entity instance that generated the macro-event; (b) `random`, a target is selected following a uniform random distribution from the pool of all active instances of the specified `type_object`; (c) `id`, the sub-action is statically bound to a specific instance identifier (e.g., "Node_24"); (d) `intelligent`, the target resolution is delegated to the action handler itself, which leverages its context to infer the target (e.g., a `disable_node` action receiving an Application ID will dynamically discover and disable the specific node hosting that application); (e) `group`, which allows a sub-action to affect a cluster of N elements simultaneously to model a wide-spread phenomena.

The `group_config` block is required for granular configuration of the `target_resolution: group` mode. The size of the cluster (`num_elements`) can be defined either as an absolute integer or as a percentage using a floating-point value. The selection strategy of the N elements in the group is driven by the `group_config.strategy` directive, which supports explicit: (a) list selection, (b) random sampling, (c) `self_random` hybrid approach (the originating instance plus $N - 1$ random instances), or the contextual proximity model, (d) `self_proximity` and (e) `random_proximity`, to model localized phenomena.

Note that assigning `type_object: global` is strictly valid only at the level of parent macro-events defined in the `global_spawner`. In this context, the global domain acts as a central scheduling timer whose sole architectural purpose is to trigger composite workflows (`composed_of`). Conversely, within the `composed_of` block itself, assigning `type_object: global` is architecturally invalid; every sub-action must explicitly delegate execution to a specific domain manager (`user`, `app`, `graph_node`, or `graph_edge`) where the mathematical and operational action handlers reside. Furthermore, when configuring ex novo instantiation actions (such as `new_user` or `new_app`), the target resolution mechanism (`target_resolution`) must be omitted entirely, as these actions dynamically spawn new entities from scratch without referencing any pre-existing target identifiers.

The following YAML snippet illustrates the versatility of the proposed framework. Listing 21 demonstrates a Catastrophic Failure macro-event. When triggered, it intelligently disables the node related to the origin entity, and concurrently creates a popularity surge in three logically proximate applications to simulate the reallocation of user traffic.

Listing 21 YAML configuration of a catastrophic failure encapsulated event

```
global_spawner:
  actions:
    catastrophic_failure_and_surge:
      type_object: global                # Values: (user | app | global)
      action_type: catastrophic_failure # Only functional required when user or app type
                                         # object, in other case is only informative or
                                         # descriptive.

      frequency:
        type: exponential
        scale: 120

      impact:
        composed_of:
          - type_object: graph_node      # 1. Cross-entity intelligent targeting
            action_type: disable_node    # Values (user | app | graph_node | graph_edge)
            impact_params:               # Values listed in Table B2
              target_resolution:
                mode: intelligent        # Values (self | random | id | intelligent | group)
                id: "Node_67"           # Only for mode:id
              distribution_to_enable_node:
                type: uniform
                low: 10
                high: 30
          - type_object: app             # 2. Group targeting based on logical proximity
            action_type: surge_popularity
            impact_params:
              target_resolution:
                mode: group               # Values (self | random | id | intelligent | group)
                group_config:            # Only for mode:group
                  strategy: random_proximity # Values (random | list | self_random | self_proximity
                                             # | random_proximity)

              num_elements: 3
              list:                # Only for strategy:list
                - "App_0"
                - "App_2"
                - "App_5"
            transient_prob:
              type: uniform
              low: 0.9
              high: 0.9
```

The generic nature of the composed-action definition also enables users to define multiple variants of the same event, each governed by different statistical behaviors. In this way, a single logical phenomenon can be represented through several stochastic profiles without modifying the simulator implementation. For example, Listing 23 configures two stochastic regimes for node degradation by defining two encapsulated events that both trigger the same `degrade_node` base action,

but with different occurrence frequencies and degradation intensities. This configuration generates degradation episodes with distinct stochastic characteristics.

Listing 22 YAML configuration of two independent stochastic regimes for degrade node event.

```
infrastructure:
  node:
    actions:
      low_frequent_degrade:
        frequency:
          type: exponential
          scale: 30
        impact:
          composed_of:
            - action_type: degrade_node
              type_object: graph_node
              impact_params:
                target_resolution:
                  mode: self
                target_attributes: ["cpu"]
                p_loss_dist:
                  type: uniform.
                  low: 0.10
                  high: 0.25
                distribution_to_restore_node:
                  type: exponential
                  scale: 3
              # first stochastic regime for degrade_node
              # very usual event
              # low degradation level
              # short MTTR
      high_hardlyever_degrade:
        frequency:
          type: exponential
          scale: 3000
        impact:
          composed_of:
            - action_type: degrade_node
              type_object: graph_node
              impact_params:
                target_resolution:
                  mode: self
                target_attributes: ["cpu"]
                p_loss_dist:
                  type: uniform.
                  low: 0.80
                  high: 0.95
                distribution_to_restore_node:
                  type: exponential
                  scale: 300
              # second stochastic regime for degrade_node
              # high inter-arrival time
              # high degradation level
              # long MTTR
```

C | YAML Configuration File Schema for Solver Definition

This appendix provides an exhaustive list of examples for the different ECCOS solver trigger policies. Each example illustrates how the corresponding policy is expressed in the YAML solver configuration and which auxiliary parameters are required to control when placement recalculation is executed during the simulation.

Listing 23 YAML configuration examples list for all the solver trigger policy alternatives.

```
trigger_policy:
  type: solve_all

trigger_policy:
  type: solve_none

trigger_policy:
  type: solve_every_t_seconds
  interval_seconds: 30.0

trigger_policy:
  type: solve_every_n_events
  batch_size: 25
```

```

trigger_policy:
  type: solve_on_event_types
  critical_events:
    - "disable_node"
    - "disable_edge"
    - "surge_popularity"

trigger_policy:
  type: combined
  interval_seconds: 60.0
  critical_events:
    - "disable_node"
    - "disable_edge"
    - "surge_popularity"

trigger_policy:
  type: solve_random_prob
  probability: 0.10                                # 10\% of probability to execute the solver after each event

trigger_policy:
  type: solve_time_windows
  windows:
    - [0.0, 100.0]
    - [500.0, 600.0]

trigger_policy:
  type: solve_event_index_ranges
  ranges:
    - [1, 50]
    - [200, 250]

trigger_policy:
  type: solve_custom_pattern
  pattern: [true, false, false, false]             # Executes the solver exactly once every 4 events

```